



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Documentazione delle modifiche progettuali

TASK R6: Gestire la Pagina del Profilo Giocatore

Anno Accademico 2025/2026

Gruppo D10
Giuseppe Bencivenga DE9000015
Nicolò Borrozzino DE9000007

Contents

1	Introduction	1
1.1	Approccio di Sviluppo	2
1.2	Struttura del sistema	3
2	Analisi Dei Requisiti	5
2.1	Requisiti Funzionali	5
2.2	Diagramma dei casi d'uso	6
2.3	Scenari	8
2.4	Diagramma delle Classi	14
2.5	Analisi dei Diagrammi delle Classi originali	16
2.6	Diagramma di Contesto	18
3	Progettazione della soluzione	20
3.1	Diagramma di sequenza	21
3.2	Diagramma delle classi	24
3.3	Diagramma di Deploy	29
4	Implementazione della soluzione	31
4.1	Riorganizzazione della pagina Profilo	31
4.1.1	Introduzione dei nuovi tab nel profilo utente	32

4.1.2	Modifica del comportamento dei tab	33
4.1.3	Inizializzazione variabili per il tab Achievement	35
4.1.4	Gestione del codice JavaScript nei tab	36
4.1.5	Modifica degli endpoint per le chiamate API .	37
4.2	Modifica del Profilo Utente	39
4.3	Visualizzazione follower/following	46
4.4	Follow/Unfollow di un Utente	49
4.4.1	Visualizzazione del pulsante follow/unfollow . .	51
4.4.2	Valutazione id associato all'utente	52
4.5	Nuovo tab Leaderboard	55
4.6	Nuovo Tab Achievement	59
4.7	Aggiornamento Tab Game History	62
4.8	Gestione dell'internazionalizzazione	64
4.9	Tabella dei moduli che hanno subito modifiche	66
5	Testing	69
5.1	Gestione Follower/Following	69

Chapter 1

Introduction

Il presente elaborato descrive l'attività di progettazione e analisi architeturale svolta nell'ambito dello sviluppo di un modulo dedicato alla gestione delle pagine profilo utente all'interno di una piattaforma web basata su un'architettura a microservizi. Il sistema complessivo, concepito per supportare l'apprendimento del software testing attraverso dinamiche di *gaming*, presenta una struttura distribuita articolata in diversi servizi indipendenti, ognuno responsabile di specifiche funzionalità applicative.

In questo contesto, il nostro contributo si concentra sul recupero, il miglioramento e la riorganizzazione della componente dedicata ai profili utente. L'obiettivo principale è far funzionare correttamente funzionalità già presenti ma non operative, rendendo al contempo il sistema più modulare, accessibile e coerente rispetto alle altre parti della piattaforma. Il lavoro si orienta quindi verso una gestione più chiara

e strutturata delle informazioni personali, del progresso dell'utente e delle interazioni con le altre funzionalità del sistema.

Il modulo da noi progettato si integra dunque all'interno di un ecosistema distribuito che sfrutta container Docker, API REST e differenti tecnologie di persistenza, mantenendo coerenza con gli obiettivi architetturali del sistema originale. Lo sviluppo del sistema di gestione del profilo si inserisce in un percorso di evoluzione della piattaforma, con la finalità di migliorarne l'usabilità, l'organizzazione dei dati e la qualità complessiva dell'esperienza utente.

1.1 Approccio di Sviluppo

Nel nostro progetto abbiamo scelto di adottare *Scrum*, un framework Agile particolarmente adatto a rilasciare rapidamente componenti funzionanti del sistema. A differenza dei metodi tradizionali, basati su pianificazioni rigide, *Scrum* consente di lavorare in modo iterativo e incrementale, focalizzandosi sulla consegna di software reale e utilizzabile.

Il progetto è stato organizzato in *Sprint*, brevi periodi di lavoro con un obiettivo specifico, al termine dei quali viene prodotto un incremento di software funzionante. Ogni Sprint ha consentito al team di ricevere feedback continuo, verificare la qualità del lavoro svolto e adattare il piano di sviluppo alle nuove esigenze emerse. Eventi come il *Planning*, il *Daily Scrum* e la *Sprint Review* hanno permesso di pianificare, sin-

cronizzare e valutare costantemente il progresso del progetto.

In sintesi, l'adozione di *Scrum* ha permesso di mantenere un ritmo sostenibile, rilasciare incrementi regolari e verificabili, e migliorare continuamente il prodotto attraverso un processo iterativo, adattivo e fortemente collaborativo.

1.2 Struttura del sistema

Nella tabella seguente viene presentata una panoramica sintetica dell'architettura su cui si è operato, con l'indicazione dei principali servizi e delle rispettive funzionalità.

In particolare, i moduli maggiormente coinvolti nel nostro lavoro sono T5, per quanto riguarda la modifica, la gestione e la riorganizzazione delle interfacce web dedicate alle informazioni del profilo utente, e T23, che contiene e gestisce i dati ad esse associati.

Servizio	Descrizione
T1	Gestisce le funzioni amministrative relative alla configurazione delle classi Java e alla manutenzione dei robot impiegati nel sistema.
T23	Fornisce i meccanismi di autenticazione, registrazione e amministrazione dei profili utente.
T4	Registra l'andamento delle partite e mantiene le informazioni fondamentali sul loro stato.
T5	Implementa la logica di gioco e mette a disposizione le interfacce web rivolte ai giocatori.
T7	Compila i test prodotti dagli utenti e valuta la coverage tramite l'utilizzo di JaCoCo.
T8	Esegue la compilazione dei test come T7, ma sfrutta gli algoritmi generativi messi a disposizione da EvoSuite.
T0	Genera automaticamente test JUnit4 tramite Randoop ed EvoSuite, fornendo materiale di supporto e baseline di confronto.
db-backup	Automatizza la creazione periodica dei backup dei database utilizzati dall'intero sistema.

Table 1.1: Descrizione sintetica dei servizi del sistema

Chapter 2

Analisi Dei Requisiti

2.1 Requisiti Funzionali

La seguente tabella riporta i requisiti funzionali relativi alla gestione e visualizzazione del profilo giocatore all'interno del sistema. Tali requisiti definiscono le funzionalità minime che l'applicazione deve garantire per permettere all'utente di consultare le informazioni personali, accedere ai contenuti dinamici del profilo e interagire con altri giocatori.

ID	Requisito Funzionale
RF01	Il sistema deve permettere la visualizzazione dell'Avatar del giocatore.
RF02	Il sistema deve permettere la visualizzazione della Bio del giocatore.
RF03	Il sistema deve implementare un selettore per la visualizzazione di una parte variabile di informazioni.
RF04	Il sistema deve permettere la visualizzazione, nella parte variabile, del riepilogo delle statistiche.
RF05	Il sistema deve permettere la visualizzazione, nella parte variabile, della classifica dei giocatori.
RF06	Il sistema deve permettere la visualizzazione, nella parte variabile, della Game History.
RF07	Il sistema deve implementare la funzionalità di <i>follow</i> per un altro utente.
RF08	Il sistema deve implementare la funzionalità di <i>unfollow</i> per un altro utente.
RF09	Il sistema deve permettere la visualizzazione del numero dei <i>followers/following</i> .
RF10	Il sistema deve permettere la visualizzazione della lista dei <i>followers/following</i> .

Table 2.1: Requisiti Funzionali

2.2 Diagramma dei casi d'uso

Di seguito viene presentato il diagramma dei casi d'uso, che illustra le principali interazioni tra l'*User* e il sistema nella gestione e visualizzazione del proprio profilo.

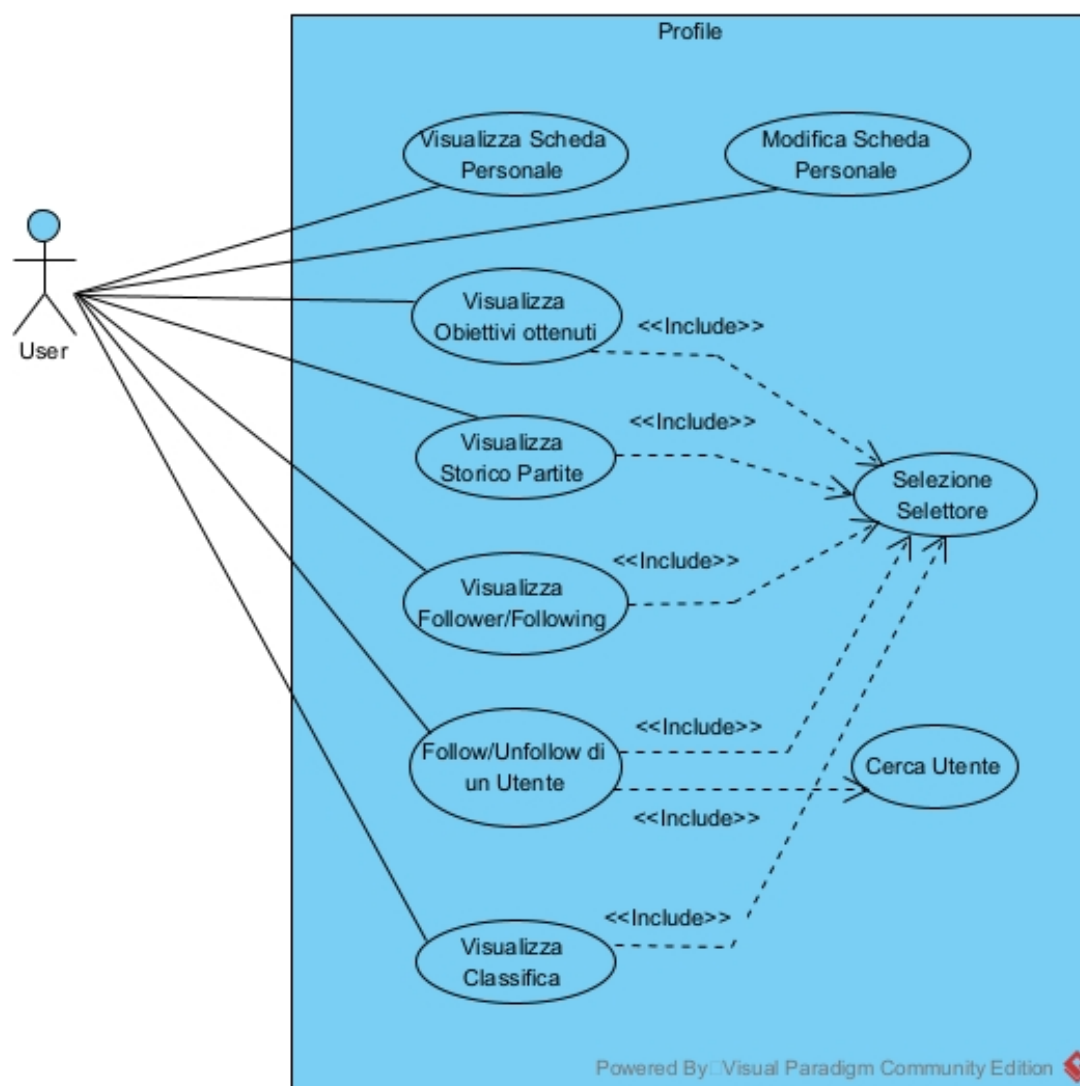


Figure 2.1: Diagramma dei Casi D’Uso

Al di fuori della visualizzazione e modifica della scheda principale, contenente le informazioni di base come *nickname*, nome, cognome e *avatar*, tutti gli altri casi d’uso presentano una relazione di tipo «include» con la selezione del selettore. Questa relazione riflette la necessità di effettuare una scelta preliminare prima di poter accedere alle informazioni specifiche desiderate.

2.3 Scenari

Visualizza Scheda Profilo	
Attore	User
Descrizione	Il caso d'uso inizia quando <i>User</i> desidera visualizzare nome, cognome, nickname, avatar e biografia
Pre-condizioni	-
Sequenza eventi	1) L'utente richiede al sistema la visualizzazione del profilo 2) Il sistema mostra le informazioni associate al profilo utente
Post-condizioni	-
Casi d'uso correlati	-
Sequenza alternativa	-

Table 2.2: Caso d'uso: Visualizza scheda personale

Modifica Scheda Profilo	
Attore	User
Descrizione	L'utente desidera modificare il nickname, l'avatar o la biografia
Pre-condizioni	-
Sequenza eventi	1) Il caso d'uso inizia quando <i>User</i> richiede al sistema la modifica del profilo 2) Il sistema espone la funzionalità di modifica del profilo 3) L'utente aggiorna nickname e/o biografia e/o avatar 4) L'utente salva le nuove informazioni inserite 5) Il sistema aggiorna il profilo utente
Post-condizioni	L'utente visualizza il profilo aggiornato
Casi d'uso correlati	-
Sequenza alternativa	-

Table 2.3: Caso d'uso: Modifica scheda personale

Selezione selettore	
Attore	User
Descrizione	L'utente seleziona opportunamente il tab relativo alle informazioni che vuole visualizzare
Pre-condizioni	-
Sequenza eventi	1) Il caso d'uso inizia quando <i>User</i> sceglie un tab 2) Il sistema seleziona il tab da visualizzare
Post-condizioni	-
Casi d'uso correlati	Visualizza obiettivi ottenuti Visualizza Storico Partite Visualizza Follower/Following Follow/Unfollow di un utente Visualizza Classifica
Sequenza alternativa	-

Table 2.4: Caso d'uso: Selezione Selettore

Visualizza Obiettivi Ottenuti	
Attore	User
Descrizione	L'utente desidera visualizzare gli obiettivi ottenuti giocando le partite
Pre-condizioni	-
Sequenza eventi	1) « <i>include</i> » <i>Seleziona Selettore</i> 2) Il sistema restituisce la pagina relativa agli obiettivi ottenuti dall'utente
Post-condizioni	-
Casi d'uso correlati	Seleziona selettore
Sequenza alternativa	-

Table 2.5: Caso d'uso: Visualizza Obiettivi Ottenuti

Visualizza Storico Partite	
Attore	User
Descrizione	L'utente desidera visualizzare le partite giocate
Pre-condizioni	-
Sequenza eventi	1) « <i>include</i> » <i>Seleziona Selettore</i> 2) Il sistema restituisce la pagina relativa alle partite giocate dall'utente
Post-condizioni	-
Casi d'uso correlati	Seleziona selettore
Sequenza alternativa	-

Table 2.6: Caso d'uso: Visualizza Storico Partite

Visualizza Follower/Following	
Attore	User
Descrizione	L'utente desidera visualizzare la lista di follower/following
Pre-condizioni	-
Sequenza eventi	1) «include» <i>Seleziona Selettore</i> 2) Il sistema restituisce la pagina contenente l'elenco di follower/following
Post-condizioni	-
Casi d'uso correlati	Seleziona selettore
Sequenza alternativa	-

Table 2.7: Caso d'uso: Visualizza Follower/Following

Follow/Unfollow di un Utente	
Attore	User
Descrizione	L'utente desidera seguire / non seguire più un altro utente
Pre-condizioni	-
Sequenza eventi	1) «include» <i>Seleziona Selettore</i> 2) «include» <i>Cerca Utente (*)</i> 3) Il sistema espone la funzionalità di <i>follow/unfollow</i>
Post-condizioni	L'utente ha seguito / non segue più un altro utente
Casi d'uso correlati	Seleziona selettore
Sequenza alternativa	-

Table 2.8: Caso d'uso: Follow/Unfollow di un Utente

(*) *La sequenza di eventi relativa al caso d'uso Cerca Utente prevede semplicemente che l'Utente fornisca come input nome/cognome/nickname e il sistema ritorna l'Utente associato, se esiste.*

Visualizza Classifica	
Attore	User
Descrizione	L'utente desidera visualizzare la classifica
Pre-condizioni	-
Sequenza eventi	1) «include» <i>Seleziona Selettore</i> 2) Il sistema restituisce la pagina relativa alla classifica
Post-condizioni	-
Casi d'uso correlati	Seleziona selettore
Sequenza alternativa	-

Table 2.9: Caso d'uso: Visualizza Classifica

2.4 Diagramma delle Classi

Per rappresentare la struttura statica del nostro sistema, abbiamo fatto uso dei **class diagram**, uno degli strumenti fondamentali dell'**UML** (**Unified Modeling Language**). L'UML è un linguaggio standardizzato per la modellazione di sistemi software che permette di rappresentare visivamente concetti come classi, interfacce, componenti e le loro relazioni. I class diagram, in particolare, mostrano le classi o le interfacce di un sistema, i loro attributi, metodi e le relazioni tra di esse, fornendo una chiara visione dell'architettura del software. Il presente diagramma delle classi rappresenta una visione di alto liv-

ello della struttura logica del sistema, con particolare riferimento alla gestione della pagina profilo dell'utente. L'obiettivo principale del diagramma è quello di modellare le entità fondamentali coinvolte nella visualizzazione e gestione delle informazioni di un giocatore, mantenendo un adeguato livello di astrazione e senza scendere nei dettagli implementativi.

Il sistema è stato modellato attorno a tre classi principali: *Profilo*, *Statistiche* e *Partita*. La classe *Profilo* rappresenta l'utente all'interno dell'applicazione e raccoglie le informazioni anagrafiche e di personalizzazione, come nome, cognome, nickname e foto profilo. Essa costituisce il punto centrale del diagramma, fungendo da elemento di collegamento con le altre classi.

La classe *Statistiche* è associata in modo univoco al profilo e ha il compito di rappresentare i dati aggregati relativi alle prestazioni del giocatore, come il livello e i punti esperienza. La classe *Partita*, invece, modella le singole partite giocate da un utente e contiene le informazioni principali relative a ciascun incontro, quali tipologia, difficoltà, punteggio e data.

Un aspetto rilevante del diagramma è la presenza di un'associazione riflessiva sulla classe *Profilo*, che consente di rappresentare il meccanismo di *follower* e *following*. Tale associazione permette a un profilo di seguire uno o più profili e, allo stesso tempo, di essere seguito da altri utenti, modellando così una relazione multi-a-molti tra istanze della

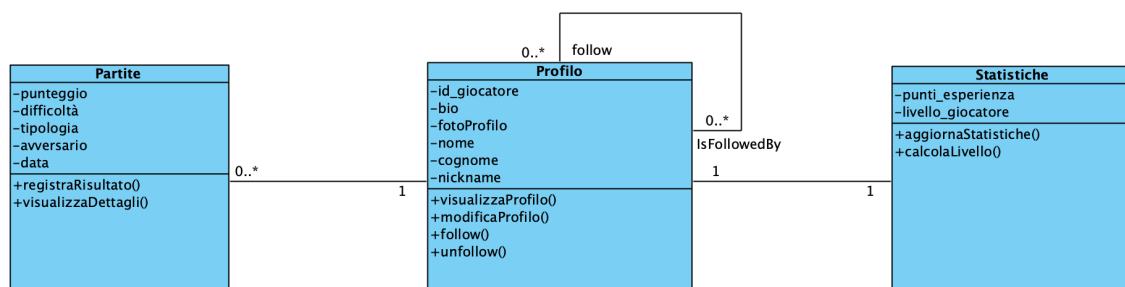


Figure 2.2: Class Diagram

stessa classe.

Il diagramma delle classi fornisce quindi una rappresentazione chiara e coerente delle relazioni tra le principali entità del sistema, costituendo una base concettuale solida per le successive fasi di progettazione e implementazione.

2.5 Analisi dei Diagrammi delle Classi originali

Durante la fase di analisi del progetto, sono stati considerati i class diagram originali presenti nella documentazione ufficiale del progetto. Questi diagrammi, sebbene non aggiornati alle ultime modifiche del codice, forniscono una rappresentazione utile della struttura iniziale del sistema, evidenziando le principali classi, interfacce e le relazioni tra di esse.

Lo scopo di inserire questi diagrammi in questa sezione è esclusiva-

mente quello di mostrare come l'architettura del sistema si sia evoluta nel tempo e quali componenti siano state aggiunte, modificate o eliminate durante lo sviluppo.

Nella sezione successiva, saranno presentati i class diagram aggiornati, completi di tutti i metodi implementati e delle modifiche introdotte durante lo sviluppo, che rispecchiano fedelmente lo stato attuale del codice.

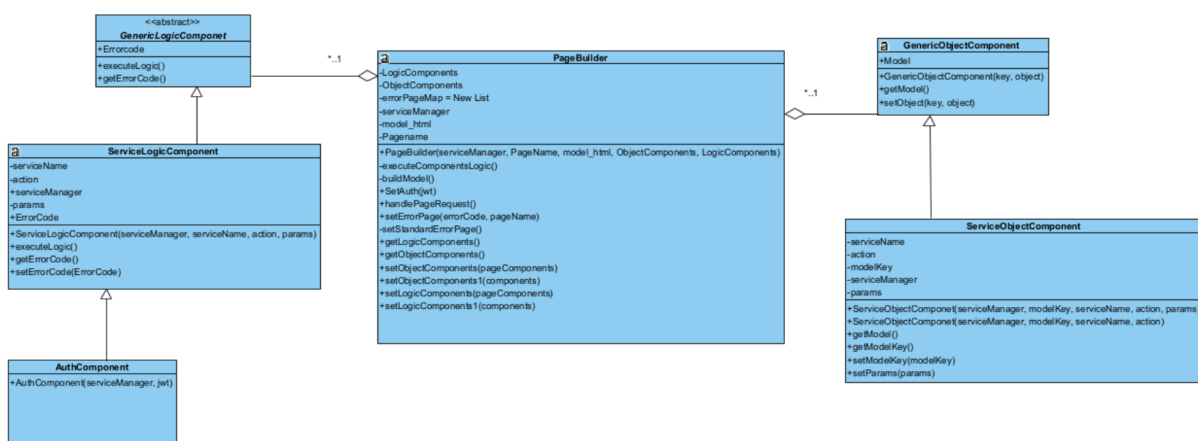


Figure 2.3: Class Diagram dei component originale

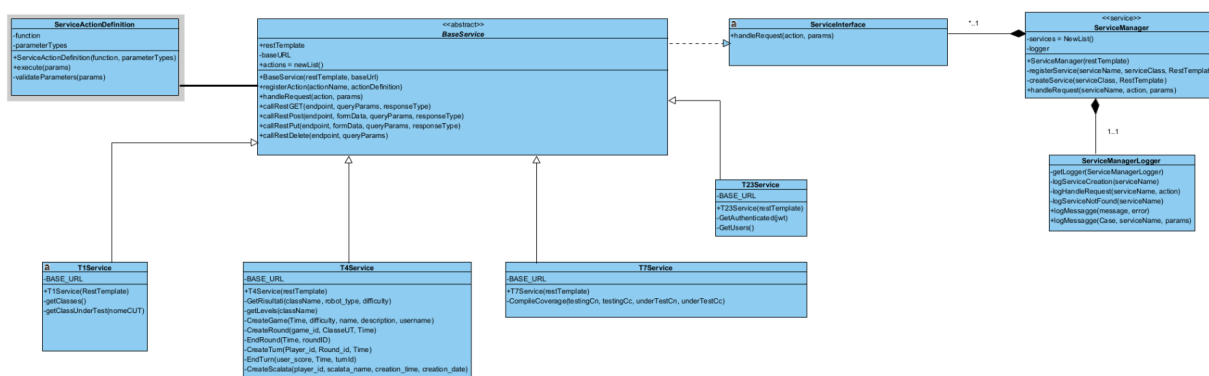


Figure 2.4: Class Diagram delle interface originale

2.6 Diagramma di Contesto

Il *Diagramma di Contesto* rappresenta una vista architeturale ad alto livello del sistema e ha l'obiettivo di fornire una comprensione immediata dei confini del dominio analizzato, evidenziando le interazioni principali tra la parte di sistema oggetto di analisi e gli attori o sistemi esterni rilevanti.

In questo documento, il diagramma di contesto è utilizzato per focalizzarsi in modo specifico sulla porzione di sistema che ci si aspetta venga impattata dalle modifiche previste. L'attenzione non è quindi rivolta all'intera architettura applicativa, ma esclusivamente ai componenti, servizi e attori esterni che interagiscono direttamente con l'area interessata, al fine di rendere esplicite le dipendenze e i punti di integrazione critici.

Le interazioni sono rappresentate in forma sintetica e intenzionale, evitando dettagli implementativi.

Il diagramma di contesto costituisce quindi il punto di partenza per l'analisi architeturale, fornendo una base condivisa su cui costruire viste di dettaglio successive (ad esempio diagrammi di sequenza).

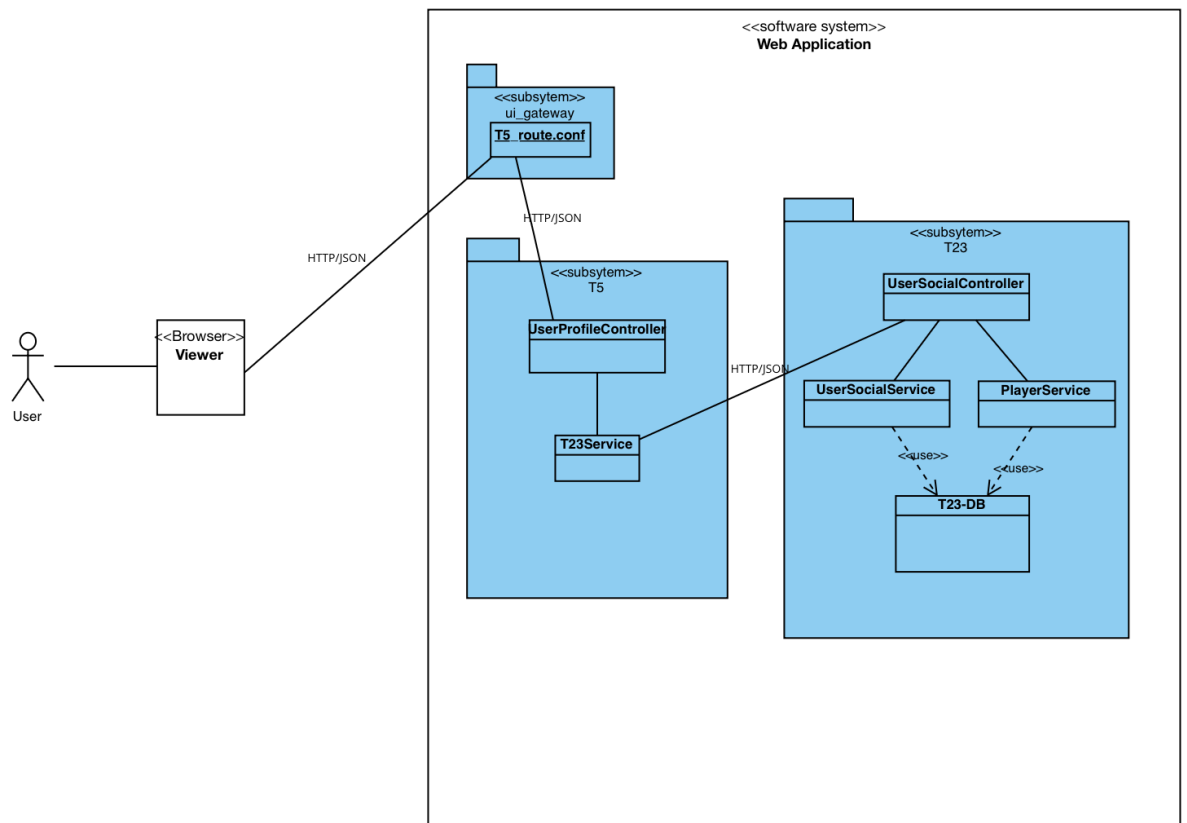


Figure 2.5: Diagramma di Contesto

Chapter 3

Progettazione della soluzione

A partire dall'analisi dei requisiti funzionali relativi al *task* assegnato, nonché dalla comprensione dell'architettura del sistema su cui si basa l'applicazione, è stata definita la soluzione progettuale adottata. Tale progettazione tiene conto delle caratteristiche del contesto applicativo e delle interazioni tra i diversi componenti coinvolti.

Nel seguito vengono presentati i più significativi diagrammi di sequenza, con l'obiettivo di descrivere il flusso delle interazioni tra i vari componenti del sistema durante l'esecuzione delle funzionalità considerate.

3.1 Diagramma di sequenza

Si precisa che la maggior parte dei nomi utilizzati per identificare i metodi presenti nei diagrammi di sequenza corrisponde a metodi effettivamente presenti nel codice. Questa scelta è stata fatta per garantire una maggiore leggibilità e per mantenere una stretta corrispondenza con l'implementazione della soluzione.

Visualizza Scheda Profilo

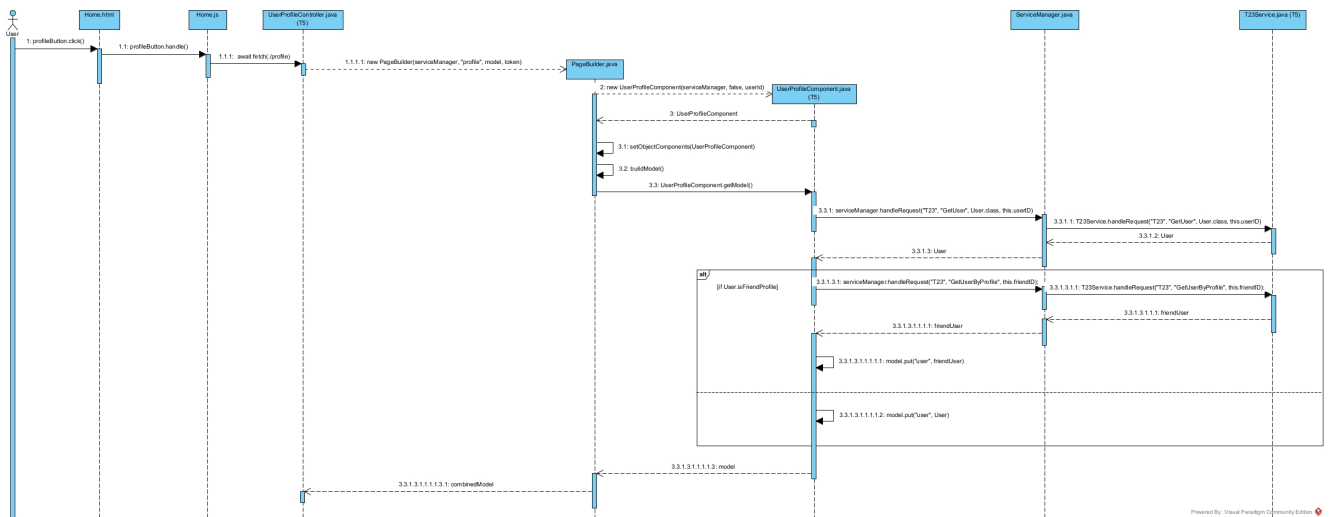


Figure 3.1: Visualizza Scheda Profilo

Nota: per evitare di appesantire ulteriormente il diagramma, si è scelto di non riportare le chiamate originate dal T23Service, in quanto si tratta esclusivamente di invocazioni di metodi che culminano nel recupero delle informazioni di interesse dal database. Esempi di chiamate di questo genere saranno riportate nei diagrammi successivi.

Modifica Scheda Profilo

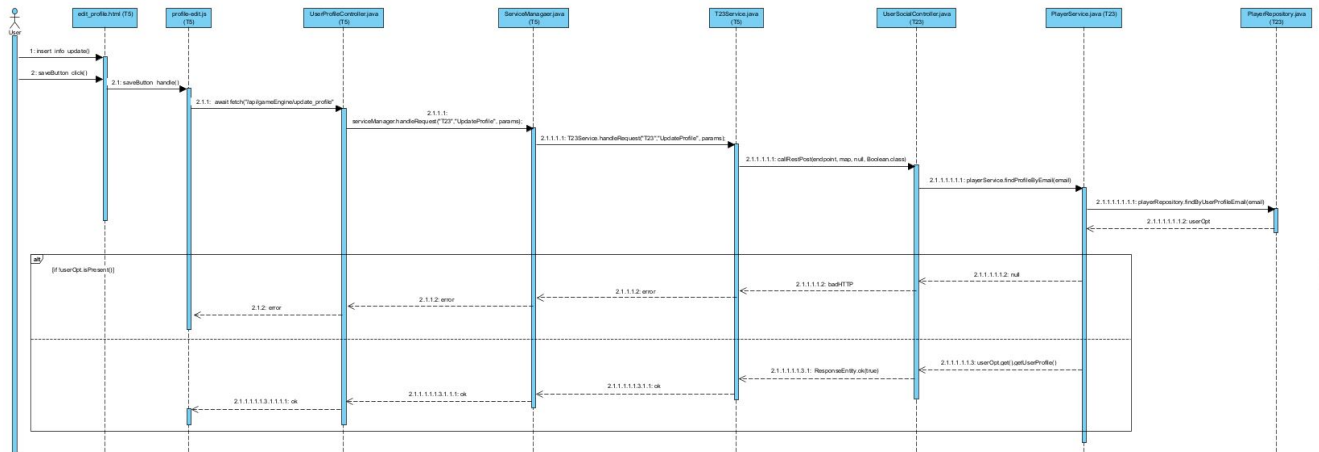


Figure 3.2: Modifica Profilo

Follow/Unfollow di un utente

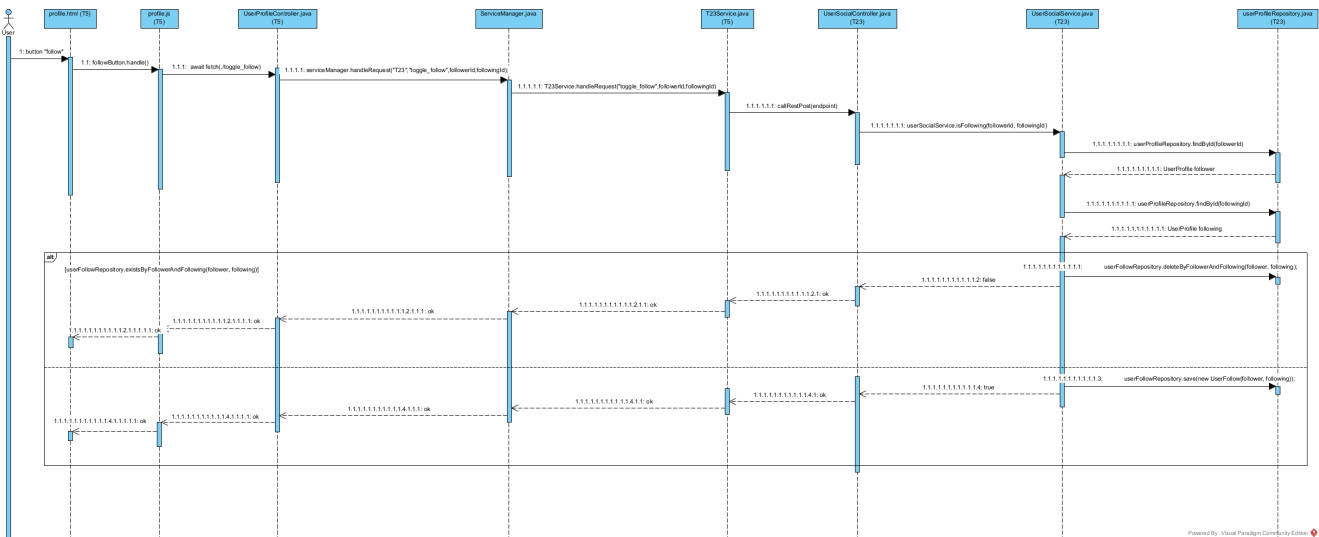


Figure 3.3: Follow/Unfollow

Visualizza Follower/Following

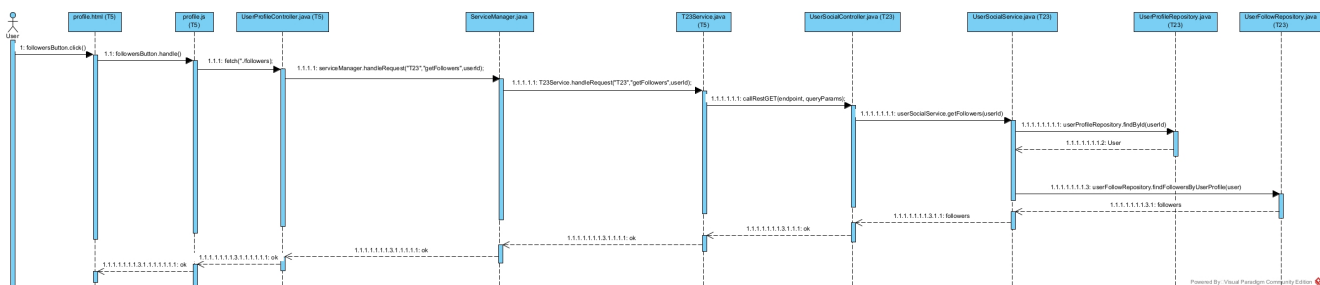


Figure 3.4: Visualizza Followers

Nota: la visualizzazione degli utenti seguiti segue un flusso del tutto analogo a quello descritto, fatta eccezione per i nomi dei metodi invocati nello specifico caso. Per tale motivo, il relativo flusso non viene ulteriormente approfondito, al fine di non appesantire la trattazione.

Visualizza Obiettivi



Figure 3.5: Visualizza Obiettivi

Visualizza Classifica

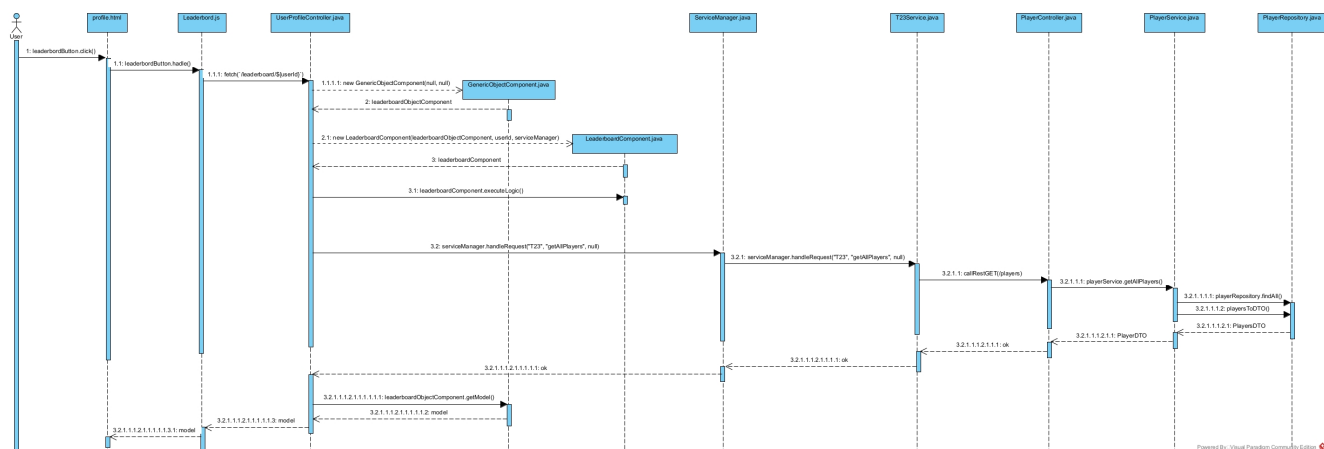


Figure 3.6: Visualizza Classifica

Visualizza Storico Partite

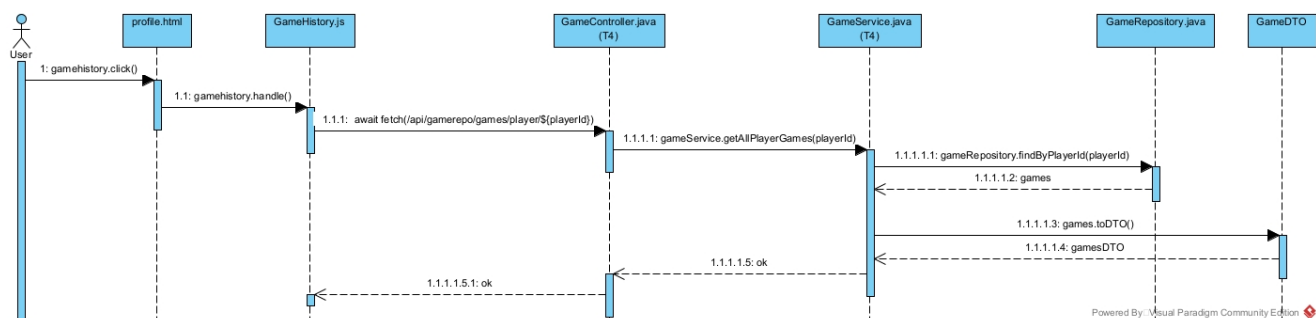


Figure 3.7: Visualizza Storico Partite

3.2 Diagramma delle classi

Nel contesto del nostro progetto, abbiamo deciso di aggiornare i class diagram precedentemente presenti nella guida ufficiale su GitHub, aggiungendo i metodi necessari per raggiungere le soluzioni implementate. Lo scopo di questo aggiornamento è stato quello di fornire una

rappresentazione completa e aggiornata dell'architettura attuale del sistema, coerente con le modifiche effettivamente implementate nel codice.

Nei diagrammi aggiornati, le modifiche rispetto alla versione originale sono evidenziate attraverso i colori:

- **Verde**: classi aggiunte nel corso dello sviluppo.
- **Rosso**: classi eliminate rispetto alla versione originale.

Legenda dei colori:

-  Classi aggiunte
-  Classi eliminate

Di seguito sono presentati i due diagrammi principali:

1. **Class Diagram delle Interface**: questo diagramma mostra le principali interfacce del sistema, le loro responsabilità e le relazioni con le classi concrete che le implementano, includendo i metodi aggiunti per rispecchiare le modifiche nel codice. L'obiettivo è evidenziare i contratti che guidano l'interazione tra le componenti software.
2. **Class Diagram dei Component**: questo diagramma illustra le classi principali dei componenti del sistema, i loro attributi, metodi e le relazioni con altre classi, includendo tutte le implementazioni aggiornate. Esso offre una visione complessiva della

CHAPTER 3. PROGETTAZIONE DELLA SOLUZIONE

struttura e delle dipendenze tra le componenti principali del progetto.

Questi diagrammi aggiornati costituiscono un riferimento chiaro e immediato per comprendere la struttura del sistema, riflettendo fedelmente le implementazioni del codice e facilitando la manutenzione, l'estensione e la collaborazione all'interno del team di sviluppo.

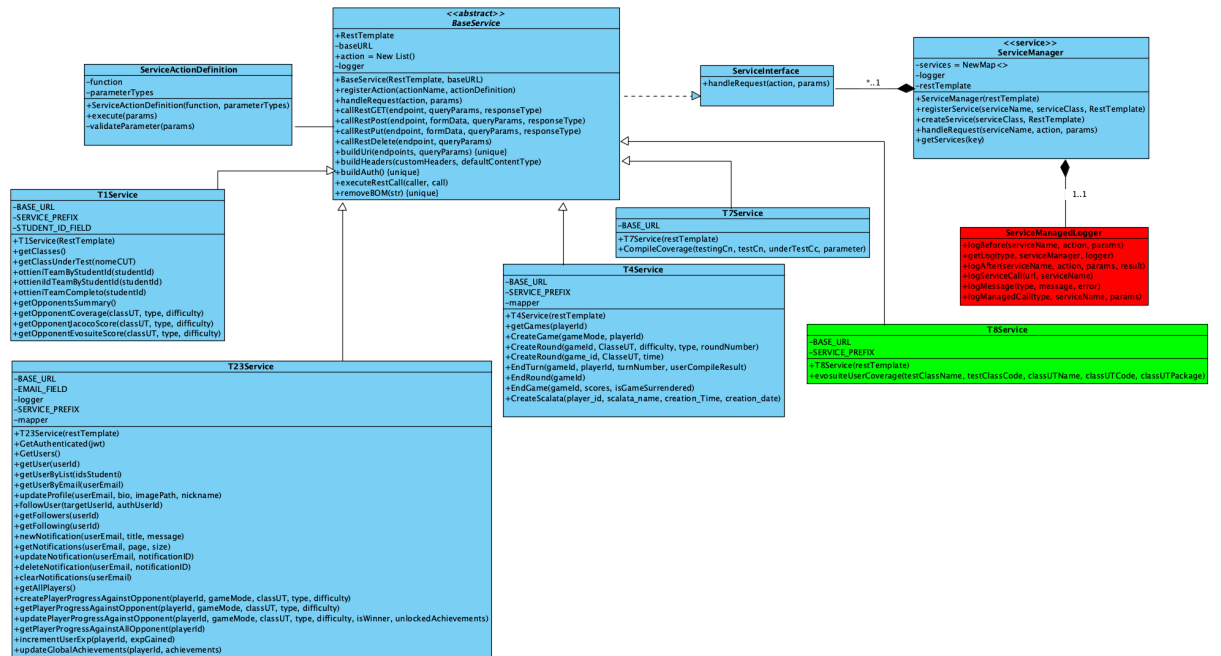


Figure 3.8: Class Diagram delle Interface (modifiche evidenziate a colori)

CHAPTER 3. PROGETTAZIONE DELLA SOLUZIONE

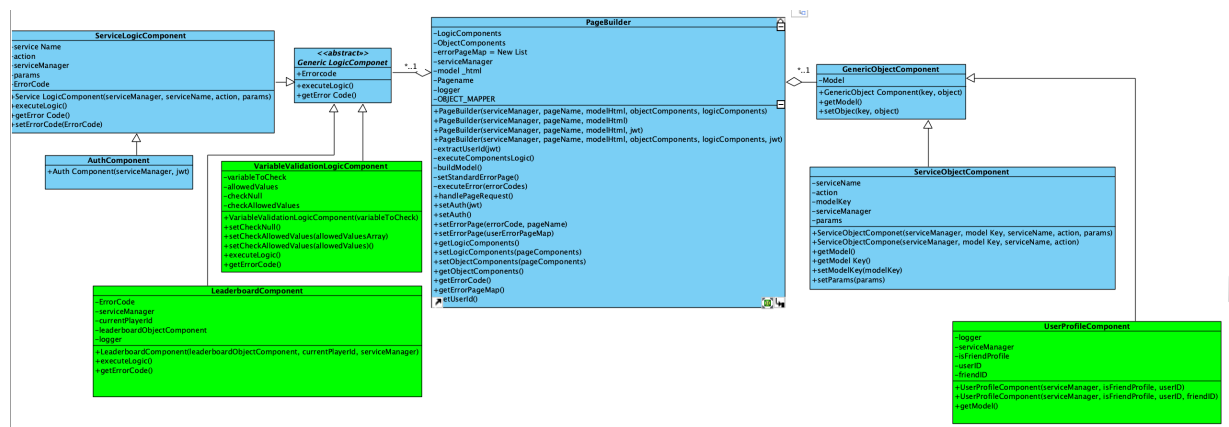


Figure 3.9: Class Diagram dei Component (modifiche evidenziate a colori)

Oltre questi due diagrammi ne viene presentato un terzo con l'intento di fornire la struttura dei controller responsabili della gestione delle rotte e della costruzione delle pagine dell'applicazione, con particolare attenzione alle funzionalità legate al profilo utente. I diagrammi dei controller sono stati aggiornati per includere tutti i metodi implementati e rispecchiano fedelmente lo stato corrente del codice.

Nel diagramma sono presenti due controller principali, `GUIController` e `UserProfileController`, entrambi responsabili della gestione delle rotte HTTP e deleganti la costruzione delle pagine al componente `PageBuilder`, scelta progettuale che consente di centralizzare la logica di composizione delle pagine, riducendo la duplicazione di codice e migliorando la manutenibilità.

Il `PageBuilder` non rappresenta un controller, ma un componente di supporto utilizzato dai controller tramite una relazione di uso (dipendenza); i controller creano istanze di `PageBuilder` esclusivamente all'interno

dei metodi di gestione delle rotte, evidenziando così una dipendenza temporanea e contestuale, correttamente modellata nel diagramma tramite frecce tratteggiate «use».

Il componente `ServiceManager` è condiviso tra i controller e il `PageBuilder` ed è responsabile della gestione delle chiamate ai servizi applicativi; le relazioni tra `ServiceManager` e i controller sono modellate come aggregazioni, la sua vita non è gestita dai controller e questi ultimi ne mantengono un riferimento stabile come attributo. La stessa relazione di aggregazione è presente tra `PageBuilder` e `ServiceManager`, sottolineando che il `PageBuilder` utilizza i servizi ma non ne controlla il ciclo di vita.

All'interno dell'architettura rappresentata, lo `UserProfileController` assume un ruolo particolarmente rilevante: esso gestisce la maggior parte delle rotte relative alle funzionalità modificate, incluse la visualizzazione del profilo personale e di altri utenti, il caricamento dei dati di achievement e leaderboard e la modifica del profilo utente. Nel diagramma lo `UserProfileController` appare come il nodo con il maggior numero di operazioni pubbliche e interazioni con il `PageBuilder`, evidenziando la sua centralità nella gestione della logica di presentazione delle nuove funzionalità introdotte.

La rappresentazione scelta permette di distinguere chiaramente tra dipendenze temporanee (uso) e relazioni strutturali (aggregazione), evitare una modellazione eccessivamente rigida mantenendo un buon

livello di astrazione e riflettere fedelmente le scelte implementative presenti nel codice aggiornato. In conclusione, il diagramma fornisce una visione coerente e aderente all'implementazione reale, mettendo in evidenza la struttura dei controller, il flusso di costruzione delle pagine e il ruolo centrale dello UserProfileController nel contesto delle modifiche apportate.

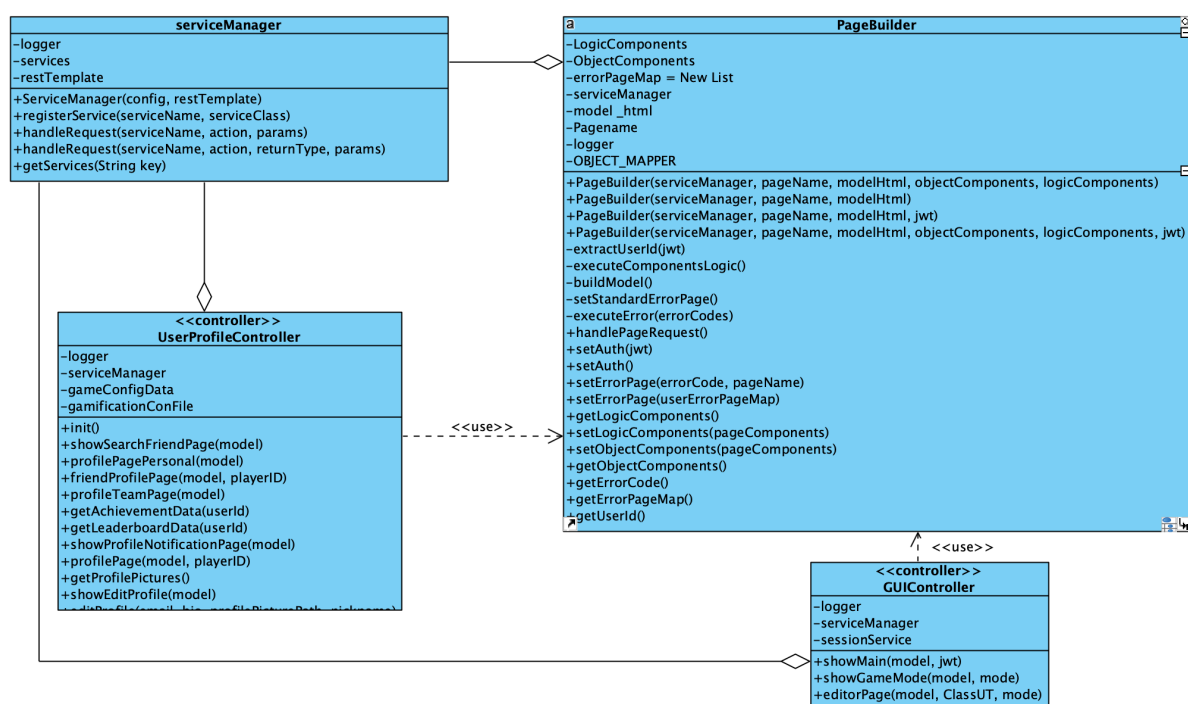


Figure 3.10: Class Diagram dei Controller (metodi e relazioni aggiornati secondo il codice corrente)

3.3 Diagramma di Deploy

Il diagramma di deploy è stato inserito con l'obiettivo di fornire una visione d'insieme chiara e immediata della struttura del progetto e delle sue componenti principali. Attraverso questa rappresentazione è

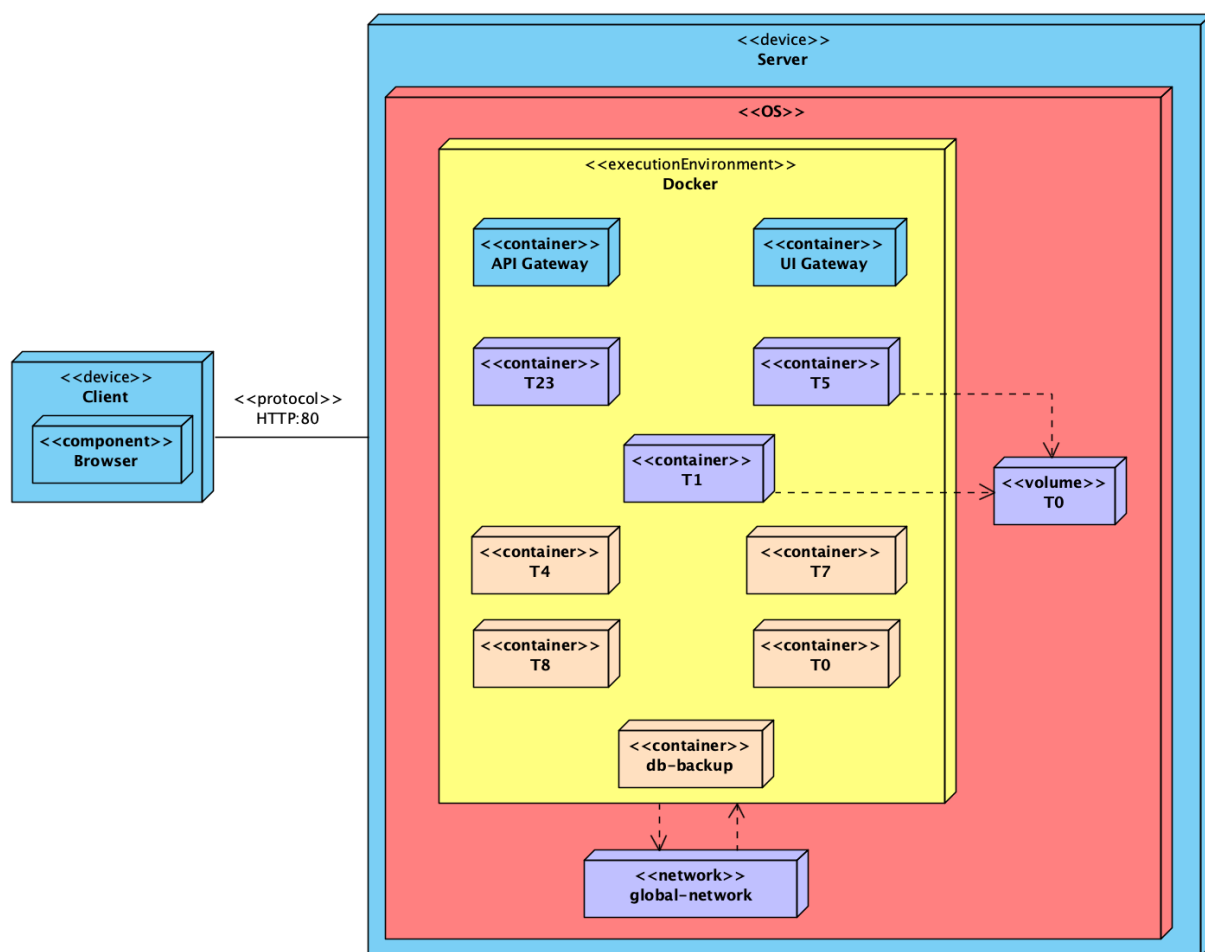


Figure 3.11: Enter Caption

possibile comprendere come i diversi elementi dell'applicazione (frontend, backend, servizi e risorse) interagiscono tra loro e su quali nodi vengono distribuiti.

Si precisa che il diagramma non è stato modificato rispetto alla versione originale del progetto, ma è stato semplicemente riportato a fini documentativi, in quanto le modifiche introdotte riguardano esclusivamente il livello di presentazione e la gestione dei contenuti nel profilo utente, senza impattare sull'architettura di deploy dell'applicazione.

Chapter 4

Implementazione della soluzione

A seguito della fase di progettazione, si procede ora a illustrare i cambiamenti implementativi apportati al sistema. In particolare, vengono evidenziate le modifiche al codice preesistente, laddove è stato possibile effettuare un riuso, e le nuove aggiunte, introdotte per attivare funzionalità precedentemente non sfruttate o per ottimizzare il flusso delle chiamate tra i componenti a livello architetturale.

4.1 Riorganizzazione della pagina Profilo

Le modifiche descritte nelle sezioni successive hanno origine da un'evoluzione strutturale della pagina `profile.html`, che è stata trasformata da semplice vista riepilogativa del profilo utente a vero e proprio punto di

accesso centrale alle funzionalità dell'applicazione.

In origine, componenti come leaderboard, achievements e cronologia delle partite erano distribuiti su pagine dedicate. Questo approccio comportava una frammentazione dell'esperienza utente e frequenti cambi di pagina.

La riorganizzazione di `profile.html` ha introdotto un'architettura basata su tab, consentendo di:

- integrare funzionalità eterogenee all'interno di un'unica vista;
- demandare il caricamento dei contenuti a script JavaScript dedicati;
- effettuare il recupero dei dati in maniera asincrona;

La pagina profilo assume quindi il ruolo di contenitore, mentre le singole funzionalità vengono inizializzate dinamicamente al momento dell'interazione dell'utente con i rispettivi tab.

4.1.1 Introduzione dei nuovi tab nel profilo utente

L'integrazione delle nuove funzionalità nel profilo utente è iniziata con l'introduzione di due nuovi tab all'interno della pagina `profile.html`.

```
<li class="nav-item" role="presentation"><button class="nav-link" id="pills-
  ↳ achievements-tab" data-bs-toggle="pill" data-bs-target="#pills-achievements"
  ↳ role="tab" aria-selected="false">Achievements</button></li>
<li class="nav-item" role="presentation"><button class="nav-link" id="pills-
  ↳ leaderboard-tab" data-bs-toggle="pill" data-bs-target="#pills-leaderboard" role
  ↳ ="tab" aria-selected="false">Leaderboard</button></li>
```

4.1.2 Modifica del comportamento dei tab

Successivamente, per ciascun tab presente nella pagina profilo, è stato modificato il comportamento di caricamento dei contenuti.

```
<div class="tab-content_mt-3">

  <!-- ===== -->
  <!-- GAMES TAB (Game History) -->
  <!-- ===== -->
  <div class="tab-pane_fade_show_active" id="pills-games" role="tabpanel">

    <!-- Contenitore della storia partite (identico a GameHistory.html) -->
    <div id="game-history-container" class="list-group"></div>

    <ul id="game-history-pagination" class="pagination_justify-content-center_mt-3">
      ↪ </ul>

  </div>

  <!-- ===== -->
  <!-- SOCIAL TAB (Followers / Following) -->
  <!-- ===== -->
  <div class="tab-pane_fade" id="pills-followers" role="tabpanel">
    <div class="d-flex_align-items-center_mb-3">
      <input type="radio" class="btn-check" name="options-follow" id="
        ↪ option_Followers" checked>
      <label class="btn_me-2" for="option_Followers">Followers</label>
      <input type="radio" class="btn-check" name="options-follow" id="
        ↪ option_Following">
      <label class="btn" for="option_Following">Following</label>
      <a class="btn_btn-primary_ms-auto" href="/SearchFriend"><i class="bi_bi-
        ↪ person-add"></i></a>
    </div>
    <input type="text" class="form-control_search-chat_mb-3" id="text-srh_follow"
      ↪ placeholder="Search_Followers">

    <div class="row_row-cols-2" id="followers_block"></div>
    <div class="row_row-cols-2" id="following_block"></div>
  </div>

  <!-- ===== -->
  <!-- ACHIEVEMENTS TAB -->
  <!-- ===== -->
  <div class="tab-pane_fade" id="pills-achievements" role="tabpanel">

    <link rel="stylesheet" href="/t5/css/Achievement.css">

    <div class="achievement-wrapper">
      <!-- SEZIONE SUPERIORE: XP, PROGRESS BAR, XP NEXT -->
      <div class="d-sm-flex_align-items-center_justify-content-around_mt-3_mb-5">

        <div class="text-center">
          <h4 class="mb-0_fw-semibold_lh-1" id="user-exp"></h4>
          <p class="mb-0_fs-5" id="total-exp-label"></p>
        </div>

        <div class="progress_container" id="progress-container">
          <svg class="progress-svg" width="150" height="150" viewBox="0_0_120_120"
            ↪ ">
            <circle class="progress_background" cx="60" cy="60" r="50"></circle>
            <circle class="progress_fill" cx="60" cy="60" r="50"></circle>
          </svg>
          <div class="progress_value" id="progress-value"></div>
        </div>

        <div class="text-center">
          <h4 class="mb-0_fw-semibold_lh-1" id="exp-next"></h4>
        </div>
      </div>
    </div>
  </div>

```

CHAPTER 4. IMPLEMENTAZIONE DELLA SOLUZIONE

```
<p class="mb-0 fs-5" id="exp-next-label"></p>
</div>

</div>

<!-- SEZIONE LISTA ACHIEVEMENTS -->
<div class="container_mt-4">
  <div class="card">

    <div class="card-header_d-flex_justify-content-between_align-items-
      ↪ center">
      <span id="achievement-title"></span>

      <select id="achievement-mode" class="form-select_w-auto">
        <option value="PartitaSingola">Partita Singola</option>
        <option value="General">Generici</option>
      </select>
    </div>

    <div class="card-body_d-flex_flex-wrap" id="achievements-container"></
      ↪ div>
    </div>
  </div>
</div>

</div>

<!-- ===== -->
<!-- LEADERBOARD TAB -->
<!-- ===== -->
<div class="tab-pane_fade" id="pills-leaderboard" role="tabpanel">

  <div class="mb-3">
    <label id="scoreTypeLabel" for="scoreTypeSelector"></label>
    <select id="scoreTypeSelector" class="form-select_w-auto">
      <option value="exp" selected></option>
      <option value="wins"></option>
    </select>
  </div>

  <div class="table-responsive_leaderboard-table">
    <table id="leaderboardTable" class="table_table-dark_table-striped_text-white">
      <thead>
        <tr>
          <th>#</th>
          <th>Nome</th>
          <th>Cognome</th>
          <th>Email</th>
          <th>Punteggio</th>
        </tr>
      </thead>
      <tbody>
        <!-- Le righe verranno popolate da leaderboard.js -->
      </tbody>
    </table>
  </div>

  <!-- Paginazione -->
  <ul id="pagination" class="pagination_justify-content-center_mt-2"></ul>

</div>

</div>
</div>
```

4.1.3 Inizializzazione variabili per il tab Achievement

Per supportare il corretto funzionamento del tab *Achievement* all'interno della pagina profilo, alcune informazioni vengono inizializzate direttamente in `profile.html` tramite Thymeleaf e rese disponibili come variabili globali JavaScript. Questa scelta consente di iniettare lato server dati di configurazione, testi tradotti e metadati degli achievement, evitando chiamate asincrone aggiuntive e sfruttando il meccanismo di internazionalizzazione già presente nell'applicazione. Le variabili così definite sono accessibili dal file `Achievement.js`, che le utilizza per il rendering dinamico dell'interfaccia, mentre i dati strettamente dinamici vengono recuperati tramite `fetch` da una rotta dedicata del backend. Questa sezione è utile al sistema per garantire che il Tab Achievement supporti la visualizzazione in più lingue.

```
<script th:inline="javascript">
  // DATI UTENTE E CONFIGURAZIONE (da Thymeleaf)
  window.ACH_userExp = /*[${userCurrentExperience}]*/ 0;
  window.ACH_expPerLevel = /*[${expPerLevel}]*/ 0;
  window.ACH_startingLevel = /*[${startingLevel}]*/ 0;
  window.ACH_maxLevel = /*[${maxLevel}]*/ 0;

  // ACHIEVEMENTS SBLOCCATI
  window.ACH_gamemode = /*[${gamemode_achievements}]*/ [];
  window.ACH_general = /*[${general_achievements}]*/ [];

  // LABEL TRADOTTE
  window.ACH_labels = {
    total_exp: /*[${total_exp_label}]*/ "Punti_esperienza_totali",
    exp_next: /*[${exp_next_level_label}]*/ "XP_per_il_prossimo_livello",
    list: /*[${achievements_list}]*/ "Achievements_ottenuti",
    single: /*[${singola}]*/ "Partita_Singola",
    general: /*[${general}]*/ "Generici",
    no_achievement: /*[${no_achievement_message}]*/ "Nessun_achievement_sbloccato"
    ↪ ,
    diff: {
      easy: /*[${d_facile}]*/ "Facile",
      medium: /*[${d_medio}]*/ "Medio",
      hard: /*[${d_difficile}]*/ "Difficile"
    }
  };

  // METADATI ACHIEVEMENTS (NOME E DESCRIZIONE)
  window.ACH_data = {
    "instructions": {
```

```

        name: /*[[#{instructions_name}]]*/ "Coverage_Master",
        descr: /*[[#{instructions_descr}]]*/ "Ottenuto_superando_il_robot_nel_numero_
            ↳ di_istruzioni_coperte."
    },
    "instructionsAndWeakMutation": {
        name: /*[[#{instructionsAndWeakMutation_name}]]*/ "Signore_delle_Weak_
            ↳ Mutation",
        descr: /*[[#{instructionsAndWeakMutation_descr}]]*/ "Superate_istruzioni_+
            ↳ weakMutation."
    },
    "firstMatchWon": {
        name: /*[[#{firstMatchWon_text}]]*/ "Primi_passi",
        descr: /*[[#{firstMatchWon_descr}]]*/ "Vinta_la_prima_partita."
    },
    "thirdMatchWon": {
        name: /*[[#{thirdMatchWon_text}]]*/ "In_ritmo",
        descr: /*[[#{thirdMatchWon_descr}]]*/ "Vinte_3_partite."
    },
    "allBeatenOneClass": {
        name: /*[[#{allBeatenOneClass_text}]]*/ "Classe_completata",
        descr: /*[[#{allBeatenOneClass_descr}]]*/ "Superati_tutti_gli_avversari_di_
            ↳ una_classe."
    },
    "allBeatenTwoClass": {
        name: /*[[#{allBeatenTwoClass_text}]]*/ "Doppia_padronanza",
        descr: /*[[#{allBeatenTwoClass_descr}]]*/ "Superati_gli_avversari_di_due_
            ↳ classi."
    }
};
</script>

```

4.1.4 Gestione del codice JavaScript nei tab

Nei nuovi tab introdotti nella pagina profilo, la logica di caricamento e rendering dei contenuti è stata quasi completamente spostata nei rispettivi file JavaScript dedicati, eliminando così la presenza di codice inline all'interno del file `profile.html`.

Un'eccezione è rappresentata dal tab Social, in cui parte del codice JavaScript rimane ancora presente in HTML. Nonostante siano stati effettuati tentativi di refactoring, alcune problematiche tecniche ne hanno impedito lo spostamento completo senza compromettere il funzionamento della ricerca e della gestione dei follower/following.

Analogamente, il tab Games è ormai gestito principalmente dal file `GameHistory.js`, ma permangono ancora poche righe di codice nel

file HTML per motivi simili, legati al corretto caricamento e inizializzazione della lista delle partite.

```
<script th:inline="javascript">
  window.userGames = /*[{$userGames}]*/ [];
</script>

<script src="/t5/js/GameHistory.js"></script>

<script>
document.addEventListener("DOMContentLoaded", function () {
  const userIdEl = document.getElementById("userProfileID");
  if (!userIdEl) return;

  // Carica subito la cronologia partite
  loadGameHistory("#game-history-container");
});
</script>
```

In questo modo, `profile.html` diventa un contenitore dinamico dei tab, con gestione asincrona dei contenuti e la maggior parte del codice JavaScript spostata nei rispettivi file dedicati, garantendo un caricamento modulare e coerente delle funzionalità.

4.1.5 Modifica degli endpoint per le chiamate API

Durante l'analisi delle API richiamate a partire dal profilo utente, è emerso che diversi endpoint risultavano non funzionanti nonostante fossero correttamente implementati nei rispettivi microservizi. Il problema non era legato alla logica applicativa dei servizi, bensì al modo in cui le richieste venivano inoltrate all'interno dell'architettura a microservizi.

In particolare, alcune chiamate venivano effettuate senza passare correttamente dall'API Gateway, oppure utilizzando rotte incomplete o non conformi alle regole di instradamento definite.

L'architettura del sistema prevede l'utilizzo di un API Gateway come

punto di ingresso unico per tutte le richieste provenienti dai client. Il gateway utilizza il path della richiesta per determinare il microservizio di destinazione, richiedendo la presenza di uno specifico prefisso per ciascun servizio.

In assenza di tale prefisso, la richiesta non veniva intercettata dal gateway e il routing verso il microservizio corretto falliva. Di conseguenza, endpoint esistenti risultavano irraggiungibili o non funzionanti dal punto di vista del client.

Il gateway utilizza il prefisso per identificare il servizio di destinazione e successivamente rimuove tale prefisso prima di inoltrare la richiesta al backend, tramite il filtro *RewritePath*. In questo modo il prefisso è utilizzato esclusivamente a livello di gateway e non influisce sugli endpoint esposti dal microservizio.

La soluzione ha previsto il riallineamento delle chiamate alle API con le regole di routing definite nell'API Gateway. In particolare:

- sono stati corretti i path delle richieste client, includendo il prefisso richiesto dal gateway;
- è stato verificato il corretto funzionamento delle regole di *RewritePath*;
- è stata ripristinata la corretta comunicazione tra gateway e microservizi.

Questo intervento ha consentito di riattivare API già presenti ma precedentemente non funzionanti, senza modificare la logica interna

dei servizi.

4.2 Modifica del Profilo Utente

La funzionalità relativa alla modifica del profilo personale da parte dell'utente è stata la prima ad essere rivisitata, con l'obiettivo di garantire almeno un livello minimo di personalizzazione delle informazioni personali all'interno del sistema. In particolare, la pagina del profilo utente, nella versione originale, si limitava a presentare una schermata contenente dati di default, senza possibilità di modifica. Sebbene le informazioni fossero correttamente recuperate dal database relativamente all'utente connesso, la visualizzazione mostrava esclusivamente le informazioni base dell'utente, un *default avatar*, un *default nickname* e un campo *bio* vuoto.

Ponendo l'attenzione lato *front-end*, si analizzano i file:

-> **edit-profile.html** (T5-G2\t5\src\main\resources\templates\Edit_Profile.html)

```
document.addEventListener("DOMContentLoaded", function () {  
    const profilePictures = document.querySelectorAll(".profile-picture");  
    const bioInput = document.getElementById("bio-input");  
    const saveButton = document.getElementById("save-button");  
  
    // MODIFICATO (player al posto di user)  
    const userData = document.getElementById("player-data");  
  
    // MODIFICATO (player al posto di user)  
    const userEmailElement = document.getElementById("player-email");  
  
    // MODIFICATO: input hidden email per UserUtil.js  
    const emailInput = document.getElementById("email");  
  
    // MODIFICATO: div notifications per notification.js  
    const notificationsDiv = document.getElementById("notifications");
```

CHAPTER 4. IMPLEMENTAZIONE DELLA SOLUZIONE

```
// Ottieni i dati dell'utente
const currentBio = userData.dataset.currentBio;
const currentImage = userData.dataset.currentImage;
const userEmail = userEmailElement ? userEmailElement.textContent.trim() : null;

// Inizializza le variabili con i valori correnti
// MODIFICATO
let selectedImage = currentImage;

// AGGIUNTA: Recupera l'input nickname dall'HTML
const nicknameInput = document.getElementById("nickname-input");

// Gestione della selezione delle immagini
profilePictures.forEach((img) => {
  img.addEventListener("click", function () {
    profilePictures.forEach((img) => img.classList.remove("selected"));
    this.classList.add("selected");
    selectedImage = this.getAttribute("src").split("/").pop().replace(/-\w{32}\.
    ↪ png$/, ".png");
  });
});

// Gestione del salvataggio
if (saveButton) {
  saveButton.addEventListener("click", async function () {
    const newBio = bioInput.value.trim();

    // AGGIUNTA: Legge il valore del nickname dall'input
    const nicknameValue = nicknameInput ? nicknameInput.value.trim() : "";

    // Usa i valori correnti se non sono stati modificati
    const bioToSend = newBio || currentBio;
    const imageToSend = selectedImage || currentImage;

    try {
      const formData = new URLSearchParams();
      formData.append("email", userEmail);
      formData.append("bio", bioToSend);
      formData.append("profilePicturePath", imageToSend);

      formData.append("nickname", nicknameValue); //AGGIUNTO

      // MODIFICA (aggiunta /api/gameEngine per comunicazione da front-end)
      const response = await fetch("/api/gameEngine/update_profile", {
        method: "POST",
        headers: {
          "Content-Type": "application/x-www-form-urlencoded",
        },
        body: formData.toString(),
      });

      if (response.ok) {
        alert("Profilo_aggiornato_con_successo!");
        // Torna alla pagina precedente e ricaricala
        window.location.href = document.referrer;
        // Se document.referrer non dovesse funzionare, usa questo come
        ↪ fallback
        setTimeout(() => {
          window.location.href = "/profile";
        }, 100);
      } else {
        const errorMessage = await response.text();
        alert("Errore_nell'aggiornamento_del_profilo:_" + errorMessage);
      }
    } catch (error) {
      console.error("Errore_di_rete:", error);
      alert("Errore_di_rete._Riprova_piu_tardi.");
    }
  });
}
```

});

- **Correzione del riferimento a `userData` → `player-data`:**
l'HTML originale non conteneva `user-data`, quindi il codice originale causava errori. Ora punta correttamente all'elemento esistente, permettendo di leggere bio, immagine e altri dati dell'utente.
- **Correzione del riferimento all'email dell'utente:** l'elemento è stato aggiornato da `user-email` a `player-email`, corrispondente al pulsante presente nell'interfaccia, consentendo di recuperare l'email correttamente.
- **Aggiunta dell'input nascosto per email:** il campo `email` consente la gestione dell'email anche da altri script (es. `UserUtil.js`) senza mostrarla direttamente nell'interfaccia.
- **Correzione del div per notifiche:** il div `notifications` è collegato a `notification.js` per permettere la visualizzazione di messaggi e avvisi all'utente.
- **Inizializzazione di `selectedImage` con l'immagine corrente:** prima era `null`, causando il salvataggio di immagini vuote se l'utente non cliccava su una foto. Ora parte dall'immagine salvata nel database, evitando errori.
- **Aggiunta del recupero dell'input `nickname` dall'HTML:** permette di leggere il nickname inserito dall'utente per includerlo

nell'aggiornamento del profilo.

- **Aggiunta della lettura del valore del nickname:** il valore inserito viene letto e pulito (`trim`), usando una stringa vuota come fallback se il campo è vuoto.
- **Inclusione del nickname nei dati inviati al server:** il nickname viene aggiunto alla richiesta POST insieme a bio, email e immagine, permettendo un aggiornamento completo del profilo.
- **Modifica dell'URL della fetch per il backend:** l'endpoint è aggiornato a `/api/gameEngine/update_profile`, garantendo la corretta comunicazione tra front-end e back-end.

-> **profile-edit.js** (`T5-G2\t5\src\main\resources\static\t5\js\profile-edit.js`)

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, _initial-scale=1.0">
  <title>Edit Profile</title>
  <link rel="stylesheet" href="/t5/css/profile-edit.css">
</head>
<body>
  <!-- Header con il pulsante -->
  <header class="header">
    <a href="/profile" class="profile-link">
      <button id="player-email" class="profile-button" th:text="${player.email}">
        ↩ Torna al profilo utente</button>
    </a>
  </header>
  <!-- Dati utente nascosti -->
  <div id="player-data">
    th:data-current-bio="${player.userProfile.bio}"
    th:data-current-image="${player.userProfile.profilePicturePath}"
    hidden>
  </div>

  <!-- MODIFICA: input hidden per rendere disponibile l'email agli script esterni -->
  <input type="hidden" id="email" th:value="${player.email}">

  <!-- MODIFICA: div per le notifiche richiesto da notification.js -->
  <div id="notifications"></div>

  <!-- Contenuto principale -->
  <main class="content">

    <!-- AGGIUNTO: label per la scelta dell[U+FFFD]immagine profilo -->
```

```

<label for="profile-pictures">Immagine profilo</label>

<!-- Sezione immagini profilo -->
<div id="profile-pictures">
  
</div>

<!-- AGGIUNTO: label per la biografia -->
<label for="bio-input">Biografia</label>

<!-- Campo per la bio -->
<textarea id="bio-input" th:text="${bio}"></textarea>

<!-- AGGIUNTO: label per il nickname -->
<label for="nickname-input">Nickname</label>

<!-- AGGIUNTO: Campo per modificare il nickname -->
<input type="text" id="nickname-input" th:value="${player.userProfile.nickname}"
      ↳ placeholder="Nickname">

<!-- Pulsante per salvare -->
<button id="save-button" th:text="#{editProfile.save}">Salva</button>
</main>
<div th:replace="fragments/footer_::_footer"></div>
<script src="/t5/js/profile-edit.js"></script>
</body>
</html>

```

- **Input hidden per l'email:** permette agli script esterni (es. `profile-edit.js`) di accedere all'email dell'utente senza mostrarla nell'interfaccia.
- **Div per le notifiche:** collegato a `notification.js` per mostrare messaggi o avvisi all'utente.
- **Label per la scelta dell'immagine profilo:** etichetta descrittiva per l'input delle immagini, migliora l'accessibilità.
- **Sezione immagini profilo:** contiene tutte le immagini disponibili per il profilo; le immagini selezionate sono gestite dal JS per aggiornare il profilo.

- **Label e campo per la biografia:** consente all'utente di modificare la propria biografia. Il campo `textarea` viene precompilato con il valore corrente.
- **Label e campo per il nickname:** permette all'utente di modificare il proprio nickname. Precompilato con il valore corrente.
- **Pulsante di salvataggio:** trigger per il JS che invia i dati modificati al backend.

Lato *back-end*, invece:

-> **UserProfileController.java** (T5-G2\t5\src\main\java\com\g2\controllers\UserProfileController.java), di cui riportiamo solo le modifiche che coinvolgono la seguente funzionalità

```
// MODIFICA
@GetMapping("/edit_profile")
public String showEditProfile(Model model) {
    try {
        PageBuilder editProfilePage = new PageBuilder(serviceManager, "Edit_Profile",
            ↪ model, JwtRequestContext.getJwtToken());

        Long userId = editProfilePage.getUserId();
        User user = (User) serviceManager.handleRequest("T23", "GetUser", userId);

        if (user == null) {
            return "error";
        }

        List<String> images = getProfilePictures();

        editProfilePage.setObjectComponents(
            new GenericObjectComponent("player", user),
            new GenericObjectComponent("images", images)
        );

        return editProfilePage.handlePageRequest();
    } catch (Exception e) {
        e.printStackTrace();
        return "error";
    }
}
```

Mostriamo qui la modifica di tale blocco, in quanto il punto di accesso

alla vera e propria pagina di modifica del profilo. Le differenze principali sono: introduzione del try-catch per gestire eccezioni e cambio del nome dell'oggetto nel modello da user a player, coerente con front-end e JS.

```
// AGGIUNTO
@PostMapping("/update_profile")
public ResponseEntity<Boolean> editProfile(

    @RequestParam("email") String email,
    @RequestParam("bio") String bio,
    @RequestParam("profilePicturePath") String profilePicturePath,
    @RequestParam("nickname") String nickname) {

    System.out.println("[DEBUG]_update_profile_chiamato_per_email:_ " + email);

    try {
        Boolean updated = (Boolean) serviceManager.handleRequest(
            "T23",
            "UpdateProfile",
            email,
            bio,
            profilePicturePath,
            nickname
        );

        return ResponseEntity.ok(updated != null ? updated : false);
    } catch (Exception e) {
        e.printStackTrace();
        // Basta ritornare ok(false) senza usare HttpStatus
        return ResponseEntity.ok(false);
    }
}
```

Aggiunta di una nuova rotta che intercetti la richiesta di aggiornamento del profilo da parte del *front-end*. Tale metodo è coerente nella ricezione dei parametri inviati e richiama opportunamente il *Service* per smistare la richiesta.

-> **T23Service.java** (T5-G2\t5\src\main\java\com\g2\interfaces\T23Service.java)

```
// MODIFICATO
registerAction("UpdateProfile", new ServiceActionDefinition(
    params -> updateProfile((String) params[0], (String) params[1], (String)
        ↪ params[2], (String) params[3] ),
    String.class, String.class, String.class, String.class
));

// MODIFICATO
private Boolean updateProfile(String userEmail, String bio, String imagePath,
    ↪ String nickname) { // aggiunto String nickname
    final String endpoint = "/profile/update_profile";
    MultiValueMap<String, String> map = new LinkedMultiValueMap<>();
```



```
map.add(EMAIL_FIELD, userEmail);  
map.add("bio", bio);  
map.add("profilePicturePath", imagePath);  
map.add("nickname", nickname); // aggiunto  
  
return callRestPost(endpoint, map, null, Boolean.class);  
}
```

Questa modifica rende il metodo coerente con le modifiche apportate al JS e all'HTML (*nickname-input*) e permette di salvare effettivamente il *nickname* modificato dall'utente.

Le restanti chiamate che completano il flusso, visibile nel diagramma di sequenza visto in fase di progettazione, sono state riutilizzate.

4.3 Visualizzazione follower/following

In questa sezione vengono descritti i cambiamenti apportati al sistema per consentire all'utente la visualizzazione degli elenchi dei profili seguiti e dei propri follower.

Per questa funzionalità, il lavoro svolto ha riguardato principalmente il riadattamento di componenti già esistenti, al fine di garantire un comportamento coerente con il flusso definito nel diagramma di sequenza presentato in fase di progettazione.

In particolare, sebbene fossero già presenti metodi funzionanti per il recupero delle informazioni necessarie, tali metodi risultavano distribuiti in modo non uniforme all'interno del sistema. Si è quindi scelto di introdurre ulteriori metodi in punti specifici dell'architettura, così da permettere un corretto flusso delle chiamate dal front-end al back-end del medesimo microservizio, il quale si occupa successivamente di in-

vocare le funzionalità già disponibili.

I file coinvolti nella modifica sono:

-> **UserProfileController.java** (T5-G2\t5\src\main\java\com\g2\controllers\UserProfileController.java), di cui riportiamo solo le modifiche che coinvolgono la seguente funzionalità

```
// AGGIUNTO
@GetMapping("/followers")
@ResponseBody
public ResponseEntity<?> getFollowers(@RequestParam String userId) {
    try {
        Object result = serviceManager.handleRequest(
            "T23",
            "getFollowers",
            userId
        );

        return ResponseEntity.ok(result);
    } catch (Exception e) {
        e.printStackTrace();
        return ResponseEntity.ok(List.of()); // niente 500 verso front
    }
}

// AGGIUNTO
@GetMapping("/following")
@ResponseBody
public ResponseEntity<?> getFollowing(@RequestParam String userId) {
    try {
        Object result = serviceManager.handleRequest(
            "T23",
            "getFollowing",
            userId
        );

        return ResponseEntity.ok(result);
    } catch (Exception e) {
        e.printStackTrace();
        return ResponseEntity.ok(List.of()); // niente 500 verso front
    }
}
```

I due endpoint REST sono stati aggiunti a supporto delle funzionalità social dell'applicazione, con l'obiettivo di intercettare le richieste di recupero dati provenienti dal *front-end* all'interno del medesimo microservizio. A tal fine, gli endpoint delegano l'elaborazione della richiesta a un Service, incaricato di restituire le informazioni richieste.

-> **T23Service.java** (T5-G2\t5\src\main\java\com\g2\interfaces\T23Service.java),

di cui riportiamo solo le modifiche che coinvolgono la seguente funzionalità

```
// AGGIUNTO: register action
registerAction("getFollowers", new ServiceActionDefinition(
    params -> getFollowers((String) params[0]),
    String.class
));

// AGGIUNTO: register action
registerAction("getFollowing", new ServiceActionDefinition(
    params -> getFollowing((String) params[0]),
    String.class
));

// AGGIUNTO
public List<User> getFollowers(String userId) {
    final String endpoint = "/profile/followers";
    Map<String, String> queryParams = Map.of("userId", userId);

    return callRestGET(endpoint, queryParams,
        new ParameterizedTypeReference<List<User>>() {});
}

// AGGIUNTO
public List<User> getFollowing(String userId) {
    final String endpoint = "/profile/following";
    Map<String, String> queryParams = Map.of("userId", userId);

    return callRestGET(endpoint, queryParams,
        new ParameterizedTypeReference<List<User>>() {});
}
```

Entrambi i metodi fungono da livello di comunicazione tra il microservizio corrente e il servizio remoto responsabile della gestione delle relazioni sociali. In questo modo, è stato possibile esporre funzionalità preesistenti al microservizio richiedente, evitando di consentirne l'accesso diretto.

I restanti metodi caratterizzanti il flusso, visibile dal diagramma di sequenza, sono rimasti inalterati.

4.4 Follow/Unfollow di un Utente

In questa sezione vengono descritti i cambiamenti apportati al sistema per consentire all'utente di seguire / non seguire più un altro utente. Analogamente a quanto osservato per le modifiche precedenti, anche in questo caso l'attenzione è stata rivolta alla riattivazione di metodi già esistenti ma originariamente non funzionanti, nonché all'introduzione di metodi aggiuntivi necessari a garantire una comunicazione più corretta ed efficace tra il *front-end* del microservizio T5 e il relativo *back-end*.

I file coinvolti nella modifica sono:

-> **UserProfileController.java** (T5-G2\t5\src\main\java\com\g2\controllers\UserProfileController.java), di cui riportiamo solo le modifiche che coinvolgono la seguente funzionalità

```
// AGGIUNTO
@PostMapping("/toggle_follow")
@ResponseBody
public ResponseEntity<Boolean> toggleFollow(
    @RequestParam String followerId,
    @RequestParam String followingId
){
    try {
        Object result = serviceManager.handleRequest(
            "T23",
            "toggle_follow",
            followerId,
            followingId
        );

        // se null, errore logico, ritorna false
        boolean followState = result instanceof Boolean && (Boolean) result;

        return ResponseEntity.ok(followState);
    } catch (Exception e) {
        e.printStackTrace();

        // in caso di errore NON mandare 500 al frontend
        return ResponseEntity.ok(false);
    }
}
```

Sebbene differisca nella logica di funzionamento, in quanto implementa un endpoint REST che intercetta richieste di tipo POST per la gestione delle operazioni di *follow* e *unfollow*, la logica concettuale alla base rimane la stessa introdotta in precedenza. In particolare, il meccanismo è finalizzato a intercettare le richieste provenienti dal front-end per la gestione dei dati all'interno del medesimo microservizio.

-> **T23Service.java** (T5-G2\t5\src\main\java\com\g2\interfaces\T23Service.java), di cui riportiamo solo le modifiche che coinvolgono la seguente funzionalità

```
// AGGIUNTO: register action
registerAction("toggle_follow", new ServiceActionDefinition(
    params -> toggle_follow(
        (String) params[0], // followerId
        (String) params[1] // followingId
    ),
    String.class,
    String.class
));

// AGGIUNTO
public Boolean toggle_follow(String followerId, String followingId) {

    String endpoint = "/profile/toggle_follow";

    MultiValueMap<String,String> form = new LinkedMultiValueMap<>();
    form.add("followerId", followerId);
    form.add("followingId", followingId);

    return callRestPost(endpoint, form, null, Boolean.class);
}
```

Tale metodo invia una richiesta POST all'endpoint con gli ID dei due utenti (chi vuole seguire e chi si vuole seguire/non seguire più) e restituisce un valore booleano che indica il successo dell'operazione. Di nuovo, funziona come livello di comunicazione tra il microservizio corrente e quello remoto, di cui vengono richieste le funzionalità.

I restanti metodi caratterizzanti il flusso, visibile dal diagramma di sequenza, sono rimasti inalterati.

4.4.1 Visualizzazione del pulsante follow/unfollow

In questa sottosezione citiamo un'ulteriore modifica a supporto della funzionalità di Follow/Unfollow.

Nello specifico, sono stati aggiunti:

in UserProfileController.java (T5-G2\t5\src\main\java\com\g2\controllers\ UserProfileController.java)

```
// AGGIUNTO
@GetMapping("/isFollowing")
@ResponseBody
public ResponseEntity<Boolean> isFollowing(
    @RequestParam String followerId,
    @RequestParam String followingId
){
    try {
        Object result = serviceManager.handleRequest(
            "T23",
            "isFollowing",
            followerId,
            followingId
        );

        boolean value = result instanceof Boolean && (Boolean) result;

        return ResponseEntity.ok(value);
    } catch (Exception e) {
        e.printStackTrace();
        return ResponseEntity.ok(false); // NON ritornare 500 al frontend
    }
}
```

in T23Service.java (T5-G2\t5\src\main\java\com\g2\interfaces\T23Service.java)

```
// AGGIUNTO: register action
registerAction("isFollowing", new ServiceActionDefinition(
    params -> isFollowing(
        (String) params[0], // followerId
        (String) params[1] // followingId
    ),
    String.class,
    String.class
));

public Boolean isFollowing(String followerId, String followingId) {
```

```
String endpoint = "/profile/isFollowing";

return callRestGET(
    endpoint,
    Map.of(
        "followerId", followerId,
        "followingId", followingId
    ),
    Boolean.class
);
```

Lo scopo concettuale di queste aggiunte rimane sostanzialmente analogo a quanto descritto per le modifiche precedenti: garantire una corretta comunicazione tra il front-end e il relativo back-end, al fine di richiamare funzionalità offerte da un servizio remoto. In questo caso specifico, il servizio consente di verificare se un utente segue un altro a partire dai rispettivi ID. L'inserimento di questa modifica è stato qui valutato, proprio perché in base al risultato di tale verifica verrà opportunamente visualizzato un pulsante di *follow/unfollow*, il quale genererà le chiamate pertinenti trattate in questa sezione.

4.4.2 Valutazione id associato all'utente

In questa sottosezione vengono discusse ulteriori considerazioni relative ai nuovi metodi introdotti a supporto della funzionalità di *Follow/Unfollow*.

In particolare, ogni volta che un utente decide di seguire o smettere di seguire un altro utente — che per comodità definiremo "amico" da questo punto in avanti — si assume che ci si trovi sulla pagina del profilo dell'amico. Tale ipotesi implica che il sistema sia stato in grado di generare correttamente la pagina profilo corrispondente.

Dal flusso analizzato nel diagramma di sequenza, per visualizzare un profilo, lo `UserProfileController` intercetta la richiesta HTTP proveniente dal front-end, con l'obiettivo di costruire una nuova pagina. In particolare, viene recuperato il componente `UserProfileComponent`, che contiene le informazioni relative al profilo utente.

Durante il recupero di tali informazioni, è previsto un controllo, tramite un'apposita proprietà, per verificare se il profilo in costruzione corrisponda a quello di un amico. In ogni caso, è comunque necessario fornire l'ID dell'utente, dato fondamentale per ricostruire correttamente le informazioni dal database.

Questo rappresenta il punto critico: nel caso di un profilo di un amico, non è possibile ricavare l'ID direttamente dal logging dell'utente, come avviene per il profilo personale. Di conseguenza, è stato necessario introdurre nuovi metodi che permettano di ricostruire l'ID dell'utente a partire dall'ID del suo profilo, garantendo così la corretta visualizzazione delle proprie informazioni. Di seguito, il flusso aggiornato e corretto

-> **UserProfileComponent.java** (T5-G2\t5\src\main\java\com\g2\components\ UserProfileComponent.java)

```
// MODIFICATO
@Override
public Map<String, Object> getModel() {
    try {
        // Inserisce i dati del profilo utente nel modello con la chiave specificata
        User user = serviceManager.handleRequest("T23", "GetUser", User.class, this.
            ↪ userID);

        // MODIFICA: richiamata nuova action GetUserByProfile in T23
        if (this.isFriendProfile) {
```



```

        User friendUser = (User) serviceManager.handleRequest("T23", "
            ↪ GetUserByProfile", this.friendID);
        this.model.put("user", friendUser);
        this.model.put("viewID", user.getUserProfile().getId());
    } else {
        // profilo privato dell'utente
        this.model.put("user", user);
        this.model.put("viewID", null);
    }
    this.model.put("isFriendProfile", isFriendProfile);
    return this.model;
} catch (Exception e) {
    // Gestione delle eccezioni, ad esempio log dell'errore
    logger.error("[UserProfileComponent]Errore_durante_il_recupero_del_profilo_
        ↪ utente:_", e);
    return Collections.emptyMap();
}
}

```

Il ramo if relativo alla valutazione di un profilo amico, prevede ora che l'oggetto venga costruito a partire da un metodo esposto dal Service che, come già comprensibile dal nome, recupera le informazioni dell'utente grazie all'identificativo del suo profilo.

-> **T23Service.java** (T5-G2\t5\src\main\java\com\g2\interfaces\T23Service.java)

```

// AGGIUNTO: register action
registerAction("GetUserByProfile", new ServiceActionDefinition(
    params -> getUserByProfile((Long) params[0]),
    Long.class
));

private User getUserByProfile(long profileId) {
    final String endpoint = "/profile/player/by/profile/" + profileId;
    return callRestGET(endpoint, null, User.class);
}

```

Registrazione e implementazione del metodo che espone la nuova funzionalità di recupero utente da parte di un servizio remoto (T23).

UserSocialController.java (T23-G1\src\main\java\com\example\db_setup\controller\
 UserSocialController.java)

```

// AGGIUNTO
@GetMapping("/player/by/profile/{profileId}")
public ResponseEntity<?> getUserByProfile(@PathVariable("profileId") int profileId)
    ↪ {
    try {

```

```

        Player player = playerService.getPlayerByProfile(profileId);
        return ResponseEntity.ok(player); // 200 OK
    } catch (UserNotFoundException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(e.getMessage());
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Si_
        ↳ [U+FFFD]_verificato_un_errore_imprevisto.");
    }
}

```

Rotta del servizio remoto (T23) che richiede le informazione dell'utente a partire dall'ID del suo profilo.

PlayerService.java (T23-G1\src\main\java\com\example\db_setup\service\PlayerService.java)

```

// AGGIUNTO
public Player getPlayerByProfile(int profileId) {

    UserProfile profile = userProfileRepository.findById(profileId);
    if (profile == null) {
        throw new UserNotFoundException("Profilo_non_trovato");
    }

    return playerRepository.findById(profile.getUserId())
        .orElseThrow(() -> new UserNotFoundException("Player_non_trovato"));
}

```

Metodo che implementa la logica del recupero dati a partire dall'id del profilo utente.

4.5 Nuovo tab Leaderboard

In origine, la visualizzazione della leaderboard era demandata a una pagina dedicata (`Leaderboard.html`), renderizzata lato server tramite Thymeleaf. Tale pagina si occupava della costruzione completa della tabella, dell'iniezione dei dati relativi ai giocatori, delle traduzioni e delle informazioni dell'utente corrente, lasciando al file `leaderboard.js` il solo compito di gestire l'ordinamento, la paginazione e l'evidenziazione

del giocatore.

Si è scelto di eliminare la pagina dedicata alla leaderboard e di integrare tale funzionalità direttamente all'interno della pagina del profilo utente. Questo cambiamento ha richiesto un ripensamento dell'architettura di visualizzazione, in quanto la leaderboard non è più caricata come pagina autonoma, ma viene visualizzata dinamicamente all'interno di un tab del profilo.

Di conseguenza, il file `leaderboard.js` è stato profondamente modificato per assumere nuove responsabilità. In particolare, il codice JavaScript non si limita più a operare su una tabella già renderizzata lato server, ma si occupa ora di:

- recuperare i dati della leaderboard tramite una richiesta asincrona al backend;
- costruire dinamicamente la struttura HTML della tabella;
- gestire il caricamento ritardato dei dati al momento dell'apertura del tab dedicato;
- mantenere le funzionalità di ordinamento, ranking e paginazione precedentemente presenti.

Questa riorganizzazione ha reso il componente leaderboard completamente indipendente da una pagina HTML dedicata, permettendone l'integrazione diretta nella vista profilo e migliorando l'esperienza utente, evitando cambi di pagina e caricamenti non necessari. Il nuovo

`leaderboard.js` rappresenta quindi un componente più autonomo e flessibile, coerente con una struttura a singola pagina in cui i contenuti vengono caricati dinamicamente in base alle interazioni dell'utente.

-> **leaderboard.js** (T5-G2\t5\src\main\resources\static\t5\js\leaderboard.js)

```
// ----- FETCH E CARICAMENTO DINAMICO -----
async function loadLeaderboard(userId) {
  const response = await fetch(`/leaderboard/${userId}`);
  if (response.status === 401) {
    window.location.href = "/login";
    return;
  }
  const data = await response.json();
  populateLeaderboard(data.leaderboard || []);
}

// ----- POPOLAMENTO DINAMICO DELLA TABELLA -----
function populateLeaderboard(players) {
  tableBody.innerHTML = "";

  players.forEach(player => {
    const row = document.createElement("tr");
    row.dataset.email = player.email;

    row.innerHTML = `
      <td class="rank"></td>
      <td>${player.name}</td>
      <td>${player.surname}</td>
      <td>${player.email}</td>
      <td class="player-score" data-exp="${player.exp}" data-wins="${player.wins}">
        ${player.exp}
      </td>
    `;

    tableBody.appendChild(row);
  });

  // Aggiorna la leaderboard con il tipo di punteggio di default
  updateLeaderboard(window.defaultScoreType || "exp");
}

// ----- CARICAMENTO AL CLICK SUL TAB -----
const leaderboardTab = document.getElementById("pills-leaderboard-tab");
if (leaderboardTab) {
  leaderboardTab.addEventListener("shown.bs.tab", () => {
    if (leaderboardLoaded) return;
    leaderboardLoaded = true;

    const userId = document.body.dataset.userid;
    loadLeaderboard(userId); // fetch asincrona
  });
}

// ----- AGGIORNAMENTO E RANKING -----
function updateLeaderboard(scoreType) {
  const rows = Array.from(tableBody.querySelectorAll("tr"));

  // Aggiorna il punteggio visualizzato
  rows.forEach(row => {
    const scoreCell = row.querySelector(".player-score");
    if (scoreCell) scoreCell.textContent = scoreCell.dataset[scoreType];
  });

  // Ordinamento decrescente e calcolo ranking
  rows.sort((a, b) => parseInt(b.querySelector(".player-score").dataset[scoreType] ||
    ↪ 0) -
    ↪ parseInt(a.querySelector(".player-score").dataset[scoreType] || 0))
    ↪ ;

  let currentRank = 1;
}
```

```

let lastScore = null;

rows.forEach((row, index) => {
  const score = parseInt(row.querySelector(".player-score").dataset[scoreType] ||
    ↪ 0);
  if (index > 0 && score !== lastScore) currentRank++;
  lastScore = score;

  row.querySelector(".rank").textContent = currentRank;
  row.classList.toggle("highlight-row", row.dataset.email === playerEmail);

  tableBody.appendChild(row);
});

setupPagination();
showPage(1);
}

```

A seguito dell'eliminazione della pagina dedicata alla leaderboard e della sua integrazione all'interno della pagina profilo, si è reso necessario introdurre una nuova rotta lato backend in grado di fornire i dati della classifica in formato JSON. In precedenza, tali dati venivano preparati e resi disponibili direttamente dal controller associato alla vista HTML tramite rendering server-side.

Con la nuova architettura, la responsabilità della visualizzazione della leaderboard è demandata al profilo utente, che, come abbiamo visto, richiede i dati in modo asincrono tramite una chiamata HTTP effettuata da `leaderboard.js`. Per supportare questo meccanismo, è stata aggiunta una nuova rotta REST nel controller del profilo utente, incaricata di eseguire la logica di costruzione della leaderboard e di restituire il modello risultante in formato JSON.

-> UserProfileController.java (T5-G2\t5\src\main\java\com\g2\controllers\

```

UserProfileController.java)

@GetMapping("/leaderboard/{userId}")
@ResponseBody
public Map<String, Object> getLeaderboardData(@PathVariable Long userId) {

  // Componente contenitore del modello dati.
  // Viene utilizzato lo stesso oggetto impiegato in precedenza
  // nella rotta che restituiva la vista HTML della leaderboard
  GenericObjectComponent leaderboardObjectComponent =
    new GenericObjectComponent(null, null);
}

```

```
// Componente di logica responsabile della costruzione della classifica.
// Riceve l'utente corrente e il service manager per accedere ai dati
LeaderboardComponent leaderboardComponent =
    new LeaderboardComponent (
        leaderboardObjectComponent,
        userId,
        serviceManager
    );

// Esecuzione della logica applicativa
boolean ok = leaderboardComponent.executeLogic();

// In caso di errore, viene restituita una risposta JSON contenente
// un messaggio di errore, gestibile dal frontend
if (!ok) {
    Map<String, Object> error = new HashMap<>();
    error.put("error", "Unable_to_load_leaderboard");
    return error;
}

// Restituzione diretta del modello popolato in formato JSON.
// Questo modello viene poi consumato da leaderboard.js
// per la costruzione dinamica della tabella nel profilo utente
return leaderboardObjectComponent.getModel();
}
```

4.6 Nuovo Tab Achievement

Nel progetto originale, la visualizzazione degli *Achievements* era gestita tramite una pagina dedicata `Achievement.html`, con rendering lato server tramite Thymeleaf. Questo funzionamento è analogo a quello seguito precedentemente dalla pagina `Leaderboard`, che ora è inclusa nel profilo. Per questo motivo, l'approccio seguito per rendere la sezione `Achievement` un tab della pagina profilo è simile a quello della sezione precedente.

Le modifiche apportate hanno previsto la rimozione del file `Achievement.html` e la creazione di due nuovi file:

- `Achievement.js`: contiene la logica di rendering dinamico degli achievements, animazione della progress bar, raggruppamento per modalità di gioco e gestione degli eventi legati al tab

Achievements nel profilo utente.

- `Achievement.css`: contiene lo stile dedicato alla visualizzazione degli achievements, progress bar e layout degli elementi grafici.

In `profile.html`, Thymeleaf continua a iniettare alcune informazioni chiave come variabili globali JavaScript (es. `esperienza` dell'utente, `achievements sbloccati`, `label tradotte`), che rappresentano il punto di partenza dei dati lato frontend. Diversamente dal passato, la logica di rendering lato server è stata sostituita da chiamate `fetch` asincrone verso il backend per recuperare dinamicamente i dati aggiuntivi degli achievements quando l'utente apre il tab. Questo approccio permette di:

1. Caricare i dati degli achievements in maniera asincrona, evitando il rendering completo lato server e migliorando la reattività dell'interfaccia.
2. Mantenere coerenza con il resto della UI del profilo, integrando badge, progress bar e lista degli achievements sbloccati.

Inseriremo successivamente solo la parte di codice che ha subito modifiche fondamentali.

-> **Achievement.js** (`T5-G2\t5\src\main\resources\static\t5\js\Achievement.js`)

```
// -----
// AGGANCIO DEL TAB (Bootstrap shown.bs.tab)
// -----
document.addEventListener("DOMContentLoaded", () => {
  const achievementTab = document.getElementById("pills-achievements-tab");
  if (!achievementTab) return;

  // =====
  // Nuovo: Fetch dati achievements al click sul tab
  // =====
  achievementTab.addEventListener("shown.bs.tab", async () => {
    const userId = document.body.dataset.userid;

    // Chiamata asincrona al backend per ottenere gli achievements
    const response = await fetch(`/Achievement/${userId}`);

    // Gestione sessione scaduta
    if (response.status === 401) {
      window.location.href = "/login";
      return;
    }

    // Recupero dati JSON
    const data = await response.json();

    // Popola variabili globali locali e avvia il rendering
    mapAchievementData(data);
    initAchievements();
  });
});

// -----
// Mappa dati ricevuti dal backend in variabili locali
// -----
function mapAchievementData(data) {
  ACH_userExp = data.experiencePoints;
  ACH_gamemode = data.gameProgressesDTO || [];
  ACH_general = data.globalAchievements || [];
  ACH_expPerLevel = data.expPerLevel || ACH_expPerLevel;
  ACH_startingLevel = data.startingLevel || ACH_startingLevel;
  ACH_maxLevel = data.maxLevel || ACH_maxLevel;
}
```

Per supportare il caricamento asincrono degli achievements nel profilo utente, è stata introdotta una nuova rotta nel `UserProfileController.java`. Questa rotta restituisce i dati necessari in formato JSON.

-> **UserProfileController.java** (T5-G2\t5\src\main\java\com\g2\controllers\

```
UserProfileController.java)

// Rotta per recuperare i dati necessari al tab Achievements
@GetMapping("/Achievement/{userId}")
@ResponseBody
public Map<String, Object> getAchievementData(@PathVariable Long userId) {

  PlayerProgressDTO p = (PlayerProgressDTO) serviceManager
    .handleRequest("T23", "getPlayerProgressAgainstAllOpponent", userId);

  Map<String, Object> map = new HashMap<>();

  // Dati principali dell'utente
  map.put("experiencePoints", p.getExperiencePoints());
  map.put("gameProgressesDTO", p.getGameProgressesDTO());
  map.put("globalAchievements", p.getGlobalAchievements());

  // Configurazioni aggiuntive richieste da Achievement.js
  map.put("startingLevel", gameConfigData.getStartingLevel());
}
```



```
map.put("expPerLevel", gameConfigData.getExpPerLevel());  
map.put("maxLevel", gameConfigData.getMaxLevel());  
  
return map;  
}
```

4.7 Aggiornamento Tab Game History

Originariamente, la cronologia delle partite era visualizzata tramite una pagina dedicata `GameHistory.html`.

Le modifiche apportate prevedono la rimozione di `GameHistory.html` e la creazione di due nuovi file:

- `GameHistory.js`: contiene la logica di caricamento dinamico delle partite, gestione della paginazione e rendering dei template HTML.
- `GameHistory.css`: contiene lo stile dedicato per la lista delle partite e badge di vittoria/sconfitta.

Il nuovo funzionamento è simile a quello dei tab `Achievement` e `Leaderboard`:

- I dati della cronologia partite vengono recuperati tramite una chiamata asincrona al backend usando l'ID dell'utente;
- La lista delle partite viene costruita dinamicamente in HTML e popolata all'apertura del tab;
- Viene gestita la paginazione lato client per evitare di caricare troppe partite in una volta;

- Le informazioni precedentemente iniettate da Thymeleaf (game mode, difficoltà, etichette) vengono ora caricate direttamente nel JS o impostate come variabili globali nel profilo.

-> **GameHistory.js** (T5-G2/t5/src/main/resources/static/t5/js/GameHistory.

```
js)

// Recupero asincrono della cronologia partite
function fetchGameHistory(playerId) {
  return new Promise((resolve, reject) => {
    $.ajax({
      url: '/api/gamerepo/games/player/${playerId}',
      type: "GET",
      dataType: "json",
      success: response => {
        if (Array.isArray(response)) resolve(response);
        else reject("Risposta_non_valida_dall'API");
      },
      error: (xhr, status, error) => reject(error)
    });
  });
}

// Rendering dinamico delle partite e gestione della paginazione
function renderGamesPage(page = 1) {
  container.innerHTML = "";
  const start = (page - 1) * GAMES_PER_PAGE;
  const end = start + GAMES_PER_PAGE;
  const gamesToShow = allGames.slice(start, end);

  gamesToShow.forEach(game => {
    let resultText = '<span_class="badge_text-bg-secondary">N/A</span>';
    let scoreText = '';

    if (game.playerResults && Object.keys(game.playerResults).length > 0) {
      const firstUserId = Object.keys(game.playerResults)[0];
      const result = game.playerResults[firstUserId];

      if (result) {
        resultText = result.winner
          ? '<span class="badge_text-bg-success">VITTORIA</span>'
          : '<span class="badge_text-bg-danger">SCONFITTA</span>';
        scoreText = `Copertura ${result.score || 0}%`;
      }
    }

    const html = `
    <a class="list-group-item_list-group-item-action">
      <div class="d-flex_justify-content-between">
        <div>
          <h5 class="mb-1">${game.rounnds?.[0]?.classUT || "N/A"}</h5>
          <p class="mb-0">${game.gameMode || "N/A"} - ${game.rounnds?.[0]?.type
            ↪ || "N/A"} - ${game.rounnds?.[0]?.difficulty || "N/A"}</p>
          <small>${formatDate(game.closedAt)}</small>
        </div>
        <div class="text-center_p-3">
          <h4 class="mb-0_fw-semibold">${resultText}</h4>
          <small>${scoreText}</small>
        </div>
      </div>
    </a>
    `;
    container.insertAdjacentHTML("beforeend", html);
  });
  renderPaginationControls();
}
```

```
// Caricamento iniziale al DOMContentLoaded
const userId = document.body.dataset.userid;
if (userId) {
  fetchGameHistory(userId)
    .then(games => {
      allGames = games.sort((a, b) => new Date(b.closedAt) - new Date(a.closedAt));
      renderGamesPage(1);
    })
    .catch(err => {
      container.innerHTML = `<div class="text-center_p-3_text-danger">Errore nel
        ↳ caricamento delle partite</div>`;
      console.error(err);
    });
}
```

4.8 Gestione dell'internazionalizzazione

Durante lo sviluppo è stata mantenuta la possibilità di cambiare lingua tra italiano, inglese e spagnolo. Questa funzionalità consente all'utente di navigare tra le pagine dell'applicazione e visualizzare immediatamente i testi dell'interfaccia nella lingua selezionata, rendendo l'esperienza più accessibile e coerente.

Per gestire i testi dinamici delle diverse sezioni dell'applicazione, sono stati creati e aggiornati diversi oggetti JavaScript globali. Ogni oggetto raccoglie le etichette traducibili di una specifica area dell'applicazione. Ad esempio, *window.ACH_labels* contiene i testi della sezione *Achievements*, come le etichette per i punti esperienza totali, il progresso verso il livello successivo e la lista degli achievement. L'oggetto *window.ACH_data*, invece, raccoglie i nomi e le descrizioni degli achievement, anch'essi tradotti.

Allo stesso modo, la leaderboard utilizza *window.LEADERBOARD_i18n* per mostrare correttamente elementi come *experience*, *wins*, *rank_label*

e le etichette di ordinamento. La sezione social fa uso di *window.SOCIAL_i18n*, che contiene i testi per i pulsanti *Followers* e *Following*, per il pulsante “Vai al profilo” e per i placeholder di ricerca. Infine, *window.GAMES_i18n* gestisce i testi relativi alla cronologia delle partite, comprese le etichette di vittoria e sconfitta, i messaggi in assenza di partite e altre informazioni, come *Coverage*.

Tutti questi testi derivano dai file di *message properties* di Thymeleaf, come *messages_it.properties*, *messages_en.properties* e *messages_es.properties*. Ad esempio, per il testo “Punti esperienza totali” viene utilizzata la chiave `total_exp_label` presente nei file di properties. Thymeleaf fornisce così la traduzione corretta, che viene assegnata agli oggetti JavaScript e successivamente visualizzata nell’interfaccia.

Grazie a questa architettura, ogni componente della UI legge le etichette dagli oggetti JavaScript invece di utilizzare stringhe hardcoded. Ad esempio, nelle card della cronologia delle partite, anziché scrivere direttamente “Coverage”, viene utilizzato `window.GAMES_i18n.coverage`. Questo approccio consente di aggiornare automaticamente i testi quando l’utente cambia lingua, senza dover modificare l’HTML statico.

In questo modo, tutte le sezioni principali dell’applicazione — *Achievements*, *Games*, *Leaderboard* e *Social* — risultano completamente internazionalizzate. Cambiando lingua dal menu a tendina, le etichette e i messaggi si aggiornano immediatamente in italiano, inglese o spagnolo. Questo sistema centralizzato semplifica l’aggiunta

di nuove lingue e la modifica delle traduzioni, garantendo coerenza in tutta l'interfaccia.

4.9 Tabella dei moduli che hanno subito modifiche

File / Modulo	Tipo di intervento	Percorso
Achievement.html	Eliminato	T5-G2/t5/src/main/resources/template/Achievement.html
GameHistory.html	Eliminato	T5-G2/t5/src/main/resources/template/GameHistory.html
Leaderboard.html	Eliminato	T5-G2/t5/src/main/resources/template/Leaderboard.html
Navbar.html	Modificato	T5-G2/t5/src/main/resources/template/Navbar.html
edit-profile.html	Modificato	T5-G2/t5/src/main/resources/template/edit-profile.html
profile.html	Modificato	T5-G2/t5/src/main/resources/template/profile.html
search.html	Modificato	T5-G2/t5/src/main/resources/template/search.html
Achievement.css	Aggiunto	T5-G2/t5/src/main/resources/static/t5/css/Achievement.css
GameHistory.css	Aggiunto	T5-G2/t5/src/main/resources/static/t5/css/GameHistory.css

CHAPTER 4. IMPLEMENTAZIONE DELLA SOLUZIONE

File / Modulo	Tipo di intervento	Percorso
profile-edit.css	Modificato	T5-G2/t5/src/main/resources/static/t5/css/profile-edit.css
profile.css	Modificato	T5-G2/t5/src/main/resources/static/t5/css/profile.css
Achievement.js	Aggiunto	T5-G2/t5/src/main/resources/static/t5/js/Achievement.js
GameHistory.js	Aggiunto	T5-G2/t5/src/main/resources/static/t5/js/GameHistory.js
Leaderboard.js	Modificato	T5-G2/t5/src/main/resources/static/t5/js/Leaderboard.js
profile-edit.js	Modificato	T5-G2/t5/src/main/resources/static/t5/js/profile-edit.js
language.js	Modificato	T5-G2\t5\src\main\resources\static\t5\js\language.js
messagesit.properties	Modificato	T5-G2\t5\src\main\resources\lang\messages_it.properties
messagesen.properties	Modificato	T5-G2\t5\src\main\resources\lang\messages_en.properties
messageses.properties	Modificato	T5-G2\t5\src\main\resources\lang\messages_es.properties
T23Service.java	Modificato	T5-G2\t5\src\main\java\com\g2\interfaces\T23Service.java
UserProfileComponent.java	Modificato	T5-G2\t5\src\main\java\com\g2\components\UserProfileComponent.java
UserProfileController.java	Modificato	T5-G2\t5\src\main\java\com\g2\controllers\UserProfileController.java

Modifiche apportate ai file e moduli del sistema

File / Modulo	Tipo di intervento	Percorso
UserSocialController.java	Modificato	T23-G1\src\main\java\com\example\db_setup\controller\UserSocialController.java
PlayerService.java	Modificato	T23-G1\src\main\java\com\example\db_setup\service\PlayerService.java

Modifiche apportate ai file e moduli del sistema - T23

Chapter 5

Testing

Il testing è fondamentale per garantire che il software funzioni correttamente e per prevenire errori in produzione. Esso permette di identificare problemi precocemente, riducendo costi e rischi.

Nel nostro caso, abbiamo effettuato test sulle funzionalità principali, verificando la correttezza delle operazioni attraverso richieste HTTP e controllando direttamente il database per assicurarci che i dati fossero coerenti con le aspettative.

5.1 Gestione Follower/Following

Per verificare il corretto funzionamento della gestione dei follower nella nostra applicazione web, abbiamo eseguito una serie di test visivi e di controllo sul database.

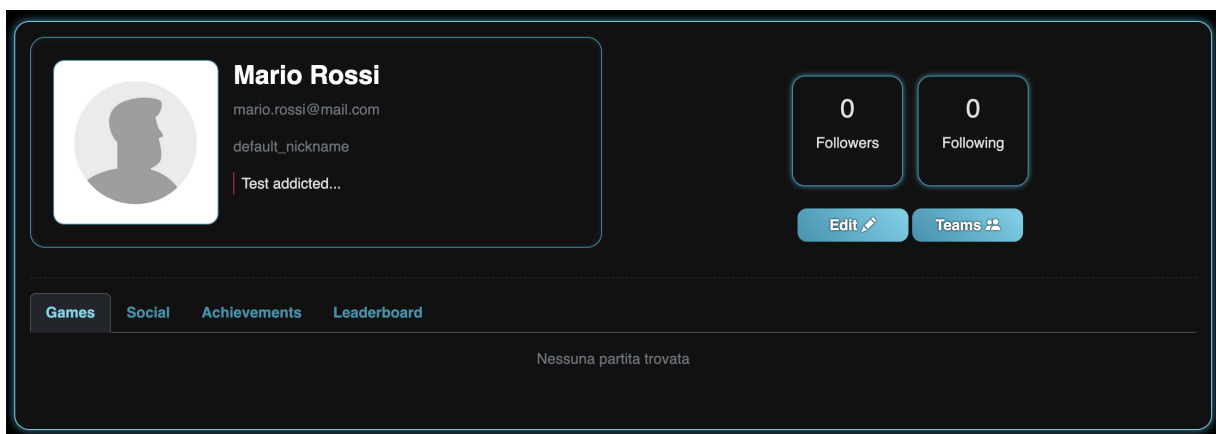


Figure 5.1: l'utente non ha follower ne following

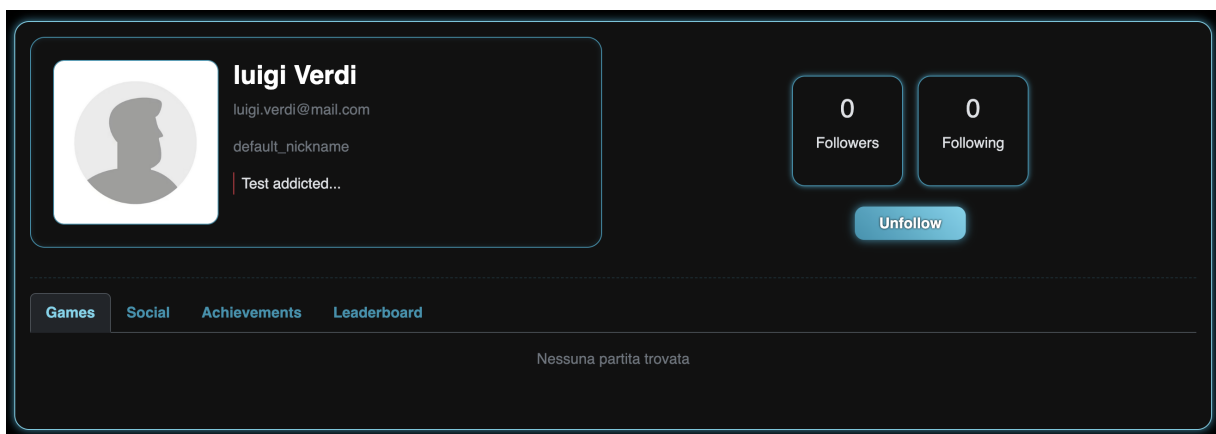


Figure 5.2: Profilo di un altro utente che permette

Stato iniziale

All'inizio, l'utente non segue nessuno. Questo è evidenziato nel seguente screenshot:

Aggiunta di un follower

Successivamente, l'utente inizia a seguire un altro utente.

L'interfaccia web viene aggiornata, mostrando il nuovo stato dei

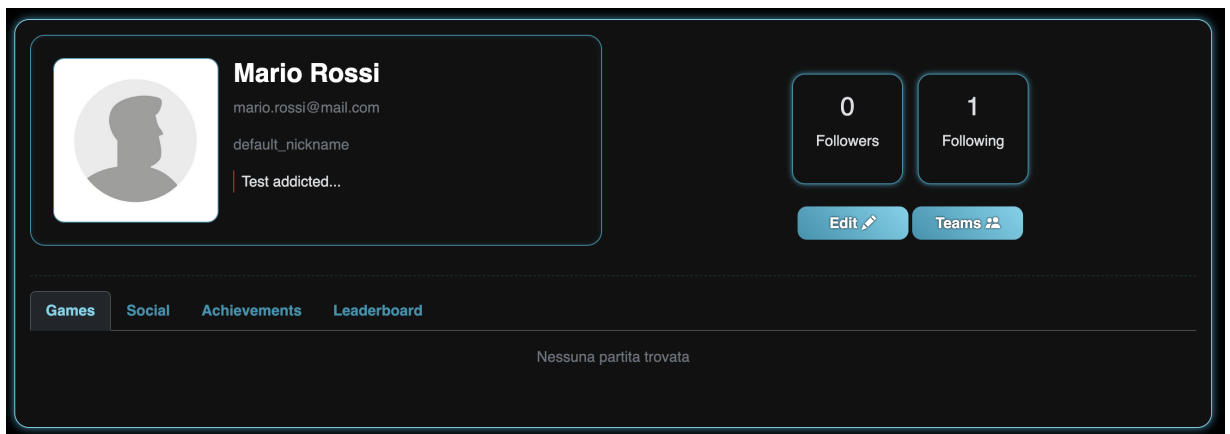


Figure 5.3: follower/following aggiornato

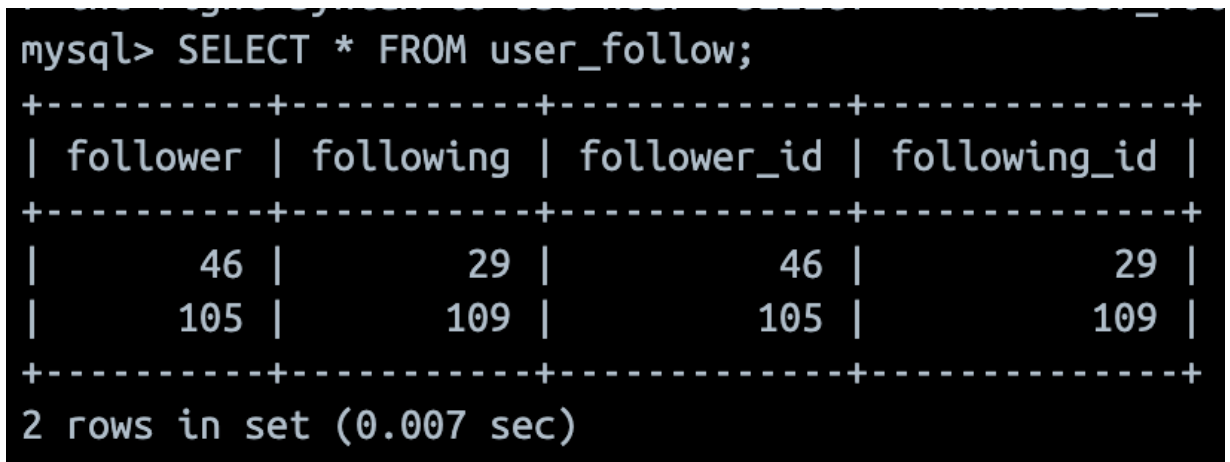


Figure 5.4: Modifica nel DB

follower/following.

Verifica nel database

Per garantire la correttezza dell'operazione, abbiamo effettuato un controllo diretto nel database, verificando che le informazioni sui follower/-following fossero aggiornate correttamente. Lo screenshot del database mostra i dati aggiornati:

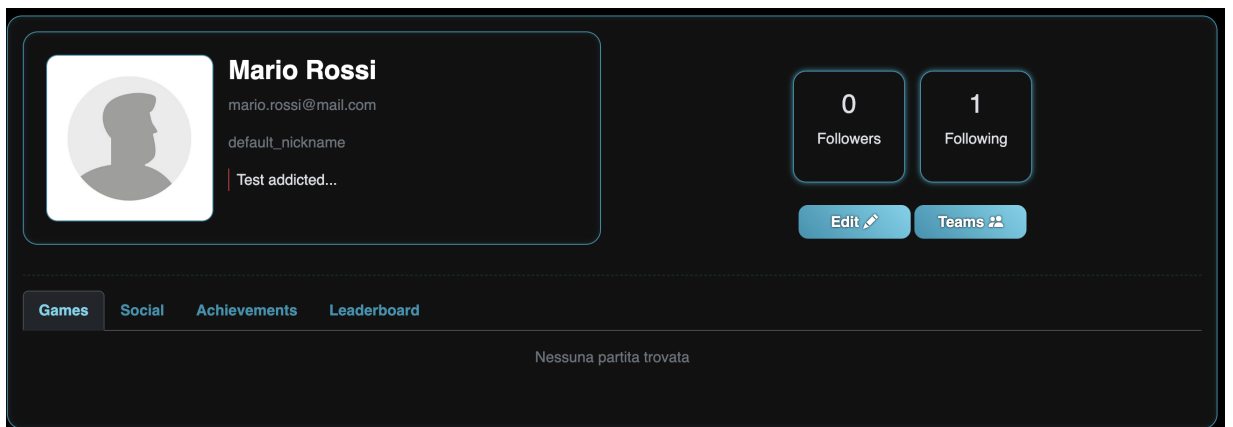


Figure 5.5: Foto iniziale

Nell'ultima immagine è possibile visualizzare che l'utente con id 105 (Mario Rossi) ha iniziato a seguire l'utente con id 109 (Luigi Verdi). Questo processo conferma il corretto funzionamento della funzionalità di gestione dei follower nella nostra applicazione.

Modifica della Foto Profilo

Per verificare il corretto funzionamento della modifica della foto profilo, abbiamo effettuato un test utilizzando la pagina `edit_profile` dell'applicazione web.

Stato iniziale del profilo

Inizialmente, l'utente presenta una determinata immagine del profilo, come mostrato nel seguente screenshot:

Nel Database presenta infatti una `default.png`.

id	bio	email	name	nickname	profile_picture_path	surname	player_id
29	ciao	nicolo.borrozzino31@gmail.com	Nicolo	default_nick	men-2.png	Borrozzino	28
46	ciao	ben.peppe@yahoo.it	Giuseppe	pepps	men-4.png	Bencivenga	45
105	Test addicted...	mario.rossi@mail.com	Mario	default_nickname	default.png	Rossi	104
109	Test addicted...	luigi.verdi@mail.com	Luigi	default_nickname	default.png	Verdi	108

Figure 5.6: Enter Caption

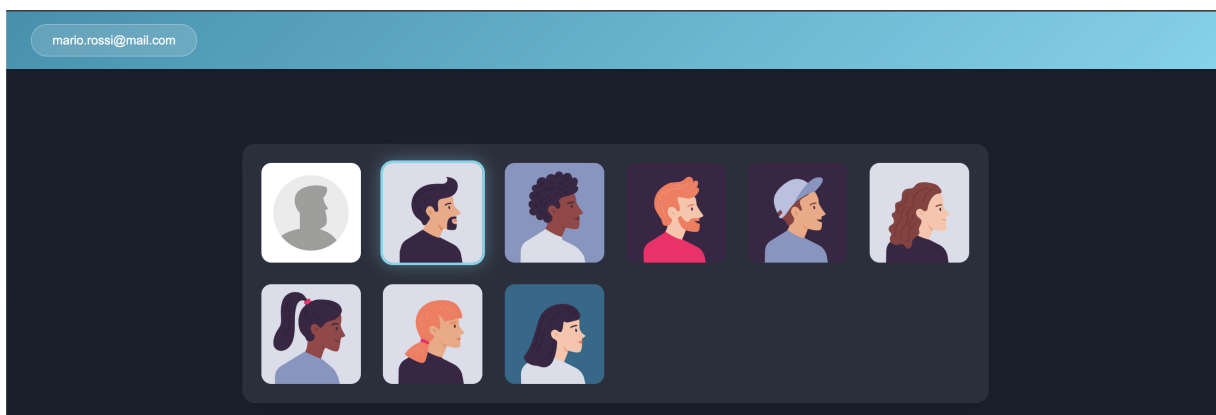


Figure 5.7: pagina Edit_profile.html

Modifica della foto profilo

Attraverso la pagina `edit_profile`, l'utente seleziona una nuova immagine e conferma la modifica.

L'interfaccia web viene aggiornata correttamente, mostrando la nuova foto profilo.

Verifica nel database

Per garantire la correttezza dell'operazione, è stato effettuato un controllo diretto sul database. Come mostrato nello screenshot seguente, il riferimento alla foto profilo dell'utente risulta correttamente aggiornato:

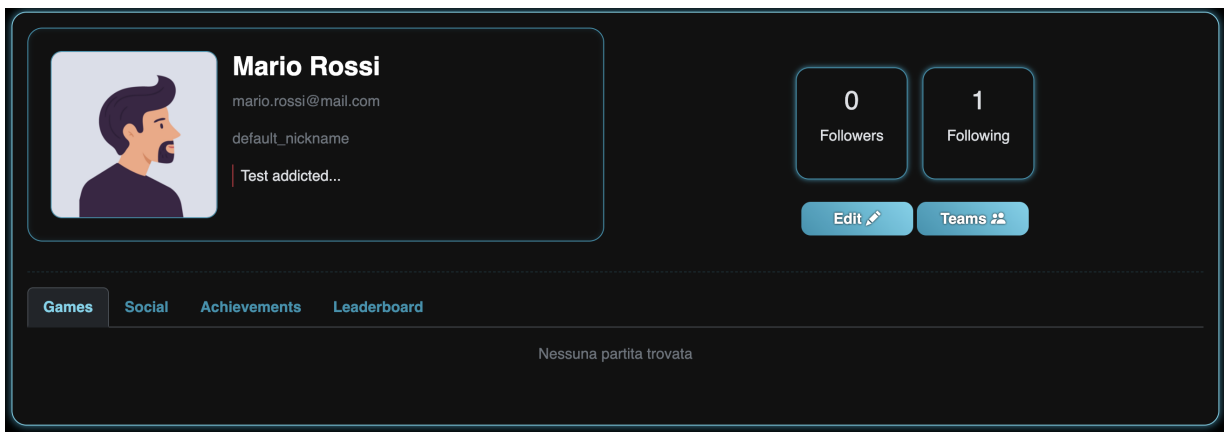


Figure 5.8: Foto modificata con successo

id	bio	email	name	nickname	profile_picture_path	surname	player_id
29	ciao	nicolo.borrozzino31@gmail.com	Nicolo	default_nick	men-2.png	Borrozzino	28
46	ciao	ben.peppe@yahoo.it	Giuseppe	pepps	men-4.png	Bencivenga	45
105	Test addicted...	mario.rossi@mail.com	Mario	default_nickname	men-1.png	Rossi	104
109	Test addicted...	luigi.verdi@mail.com	Luigi	default_nickname	default.png	Verdi	108

Figure 5.9: Foto aggiornata anche nel DB

Questo test conferma che la funzionalità di modifica della foto profilo opera correttamente sia a livello di interfaccia utente sia a livello di persistenza dei dati nel database.